

Projecte Hefesto

Miniprojectes

Mòdul M0379 — Projecte Intermodular

CFGS Administració de Sistemes Informàtics en Xarxa (ASIX2)
Laboratori d'infraestructura IT

Miniprojectes

- Gestió de projectes (33 hores)
- Creació d'Entorn Virtual amb Proxmox VE (Bàsic + Avançat - 66 hores)
- IDS / IPS amb Suricata (33 hores)
- Monitorització de xarxes amb Zabbix (33 hores)
- Kubernetes (Bàsic - 33 hores)

Autors:

Axel David Rodriguez
Daniel Pañero Juste

Tutors: Isaac, Jose, Josep Catá, Francesc
Institut Sa Palomera — Blanes — Curs 2025/2026

Repositori GitHub:

https://github.com/dpanero/Projecte_Hefesto_DPJ_ADR

Jira:

<https://axeldani.atlassian.net/jira/software/projects/PROJECTE/boards/35>

Índex

Introducció I Planificació	5
1.1 Justificació i abast del projecte	5
1.2 Equip i organització	5
1.3 Eines de gestió i treball	6
1.4 Planificació temporal	7
Tasques per sprint	7
1.5 Riscos identificats	8
1.6 Seguiment, desviacions i adaptacions	9
1.7 Retrospectives	9
Projecte Proxmox	10
1. Introducció i justificació tècnica	10
1.1 Objectiu del miniprojecte	10
1.2 Comparativa de plataformes i tria	10
1.3 Repartiment de feina	10
2. Disseny de la solució	10
2.1 Arquitectura general	10
2.2 Topologia de xarxa	11
2.3 Decisions clau	11
3. Implementació de la fase bàsica (33 h)	11
3.1 Preparació de l'entorn (VirtualBox)	11
3.2 Instal·lació de Proxmox VE 9.1	12
3.3 Configuració de xarxa al node	12
3.4 Emmagatzematge centralitzat amb TrueNAS Core	13
3.5 Plantilla d'Ubuntu Server amb cloud-init	13
3.6 Perfils de recursos i optimització	14
4. Implementació de la fase avançada (33 h)	14
4.1 Clusterització dels tres nodes	14
4.2 Servidor de backups (Proxmox Backup Server)	14
4.3 Alta disponibilitat amb Ceph	15
4.4 Replicació ZFS (alternativa lleugera a Ceph)	15
4.5 Elasticitat: Hotplug + autoescalat	15
4.6 Monitoratge integrat (clúster · node · VM)	16
5. Validació i proves	16
6. Conclusions i reflexió	17
6.1 Objectius assolits	17
6.2 Dificultats i com s'han resolt	17
6.3 Possibles millores futures	18
6.4 Reflexió personal	18

Projecte Docker-Orquestradors	20
1. Introducció i arquitectura general	20
1.1 Serveis de la plataforma	20
1.2 Fluxos funcionals (verificats a totes les fases)	20
1.3 Tecnologies	20
2. Docker Compose: Entorn de desenvolupament	21
2.1 Arquitectura de xarxes	21
2.2 Elements clau del docker-compose.yml	21
2.3 Patrons implementats als microserveis	21
2.4 Resultats i aprenentatges	21
3. Docker Swarm: Alta disponibilitat	22
3.1 Infraestructura	22
3.2 Diferències Compose → Swarm (docker-stack.yml)	22
3.3 Configuració de deploy: aplicada	22
3.4 Race condition detectada i solució	22
3.5 Demostracions realitzades	23
4. Seguretat al clúster Swarm	23
4.1 Vulnerabilitats identificades a la Fase 2	23
4.2 Docker Secrets	23
4.3 Cas especial: RabbitMQ 3.13+	24
4.4 Aïllament de xarxes overlay	24
4.5 TLS automàtic de Swarm	24
4.6 Escaneig de vulnerabilitats (Docker Scout)	25
4.7 Defensa en profunditat	26
5. Kubernetes: Entorn de producció	27
5.1 Swarm vs Kubernetes — Comparativa clau	27
5.2 Estructura de manifests Kubernetes	27
5.3 Init containers	27
5.4 Rolling update demostrat	28
5.5 Fluxos funcionals verificats	28
5.6 Conclusions de la Fase 4	28
6. Conclusions globals	29
6.1 Evolució de l'arquitectura	29
6.2 Conceptes apresos	29
6.3 Limitacions i millores futures	29
IDS/IPS amb Suricata	31
1. Objectiu del sistema IDS/IPS dins d'Hefesto	31
2. Arquitectura de xarxa i zones vigilades	32
3. IDS, IPS i criteri aplicat al projecte	33

4. Selecció de tecnologia: per què Suricata	34
5. pfSense com a base de firewall i perímetre	35
6. Instal·lació i configuració inicial de Suricata	35
7. Polítiques de seguretat per WAN, LAN i DMZ	36
8. Regles, signatures i categories utilitzades	36
9. Logs, alertes i enviament cap a Zabbix	37
10. Simulació d'atacs i comprovació de funcionament	38
11. Problemes trobats i ajustos de Suricata	38
12. Resposta a incidents i actuació pràctica	39
13. Integració amb sistemes complementaris	39
14. Validació final del sistema IDS/IPS	40
15. Resultat final, límits i millores futures	41
Monitorització amb Zabbix	42
1. Objectiu real de la monitorització a Hefesto	42
2. Arquitectura de xarxa i posició del servidor Zabbix	42
3. Instal·lació base i preparació del servidor	43
4. Configuració inicial de la interfície i criteris de treball	44
5. Alta de hosts, grups i autodescobriment	45
6. Monitorització del NAS TrueNAS per SNMP	46
7. Integració del clúster Proxmox i problema d'alertes duplicades	46
8. Recepció de logs de pfSense i Suricata amb rsyslog	47
9. Monitorització personalitzada del servidor web de la DMZ	48
10. Items, health score i resum del servei web	48
11. Triggers, llindars i reducció de soroll	49
12. Dashboards operatius i vista general del sistema	50
13. Informes periòdics, correu i problemes trobats	51
14. Proves de validació realitzades	51
15. Problemes, ajustos i resultat final	52
Bibliografia	53
Proxmox:	53
Docker:	53
Anexo de enllacs del repositori Hefesto	¡Error! Marcador no definido.

Introducció I Planificació

1.1 Justificació i abast del projecte

El projecte Hefesto neix de la voluntat de construir un laboratori d'infraestructura IT realista que reproduïxi, a escala reduïda, l'entorn tècnic d'una petita o mitjana empresa tecnològica. El nom està inspirat en el déu grec del foc i la forja, simbòlicament associat al taller on es construeixen eines: la nostra forja és el servidor físic on s'aixeca tota la infraestructura virtual del cicle.

L'objectiu és integrar en una única plataforma els coneixements adquirits durant el cicle d'Administració de Sistemes Informàtics en Xarxa: virtualització, xarxes, seguretat, monitoratge i orquestració de contenidors. En lloc de desenvolupar cada miniprojecte de manera aïllada, hem optat per un fil conductor comú una xarxa LAN/DMZ real amb domini propi i accés exterior controlat que dona coherència al conjunt.

Què es vol fer:

- Desplegar una plataforma de virtualització Proxmox sobre maquinari físic.
- Construir una xarxa segmentada (LAN i DMZ) gestionada per un firewall pfSense.
- Publicar serveis a Internet de forma segura mitjançant DDNS, domini propi i reverse proxy.
- Habilitar accés remot xifrat amb OpenVPN i certificats.
- Implementar un sistema de detecció i prevenció d'intrusions (IDS/IPS).
- Monitorar tota la infraestructura amb mètriques i alertes.
- Desplegar microserveis amb Docker i Kubernetes.

Recursos disponibles:

- Connexió a Internet domèstica amb IP pública dinàmica.
- Domini propi hefesto.digital adquirit a Nominalia.
- Llicències gratuïtes / open source (Proxmox VE, pfSense, Suricata, Zabbix, Docker, Kubernetes).
- Equip humà: 2 alumnes treballant en paral·lel.

1.2 Equip i organització

L'equip està format per dos alumnes d'ASIX2 que s'han repartit la feina seguint criteris d'especialització i interès personal:

Membre	Àrees lidera des
Daniel Pañero Juste (DPJ)	Disseny i gestió de xarxa (pfSense, OpenVPN, DDNS, reverse proxy), seguretat (Suricata) i monitoratge (Zabbix)
Axel David Rodriguez (ADR)	Plataforma de virtualització (Proxmox VE bàsic i avançat) i orquestració de contenidors (Docker, Kubernetes)

Conjunt	Gestió del projecte, disseny global d'arquitectura, integració entre miniprojectes, redacció de la memòria i defensa
---------	--

Aquest repartiment respon a una lògica de capes: Daniel s'ha ocupat del substrat de xarxa i visibilitat operativa, mentre que Axel ha treballat el substrat de còmput. D'aquesta manera, cada miniprojecte té un responsable clar, però l'arquitectura final només té sentit quan les dues capes s'integren.

La comunicació diària s'ha realitzat de manera presencial a l'aula i, fora d'aquesta, mitjançant missatgeria instantània.

4.3 Metodologia de treball: Agile adaptat

Hem adoptat una filosofia Agile com a base de la gestió, adaptada a un equip reduït de dues persones. No s'ha aplicat Scrum complet (no hi ha rols de Product Owner, Scrum Master ni client extern), però sí s'han mantingut els principis fonamentals del marc:

- Iteracions curtes (sprints) amb objectius concrets i tangibles.
- Adaptació contínua: replanificació quan apareixen bloquejos o noves dependències.
- Autonomia i responsabilitat individual dins de cada sprint.
- Revisió retrospectiva al final de cada iteració per detectar millores.

A diferència d'un Scrum convencional amb sprints de durada fixa, hem decidit fer coincidir cada sprint amb un miniprojecte. Aquesta decisió ve motivada per la naturalesa del mòdul: els miniprojectes són unitats de treball ja delimitades, amb objectius i lliurables propis, i tractar-los com a sprints permet aplicar la lògica Agile sense forçar artificialment el calendari.

1.3 Eines de gestió i treball

Funció	Eina	Justificació
Repositori de codi i documentació	GitHub (privat amb col·laborador projectesasixsapalomera)	Estàndard del sector, control de versions i visibilitat per al professorat
Task board / gestió de tasques	Jira	Eina professional àmpliament utilitzada al sector; permet definir sprints, històries i tasques amb estats
Comunicació	[!REVISAR: WhatsApp/Discord]	Missatgeria instantània per coordinació diària
Documentació tècnica	Markdown (al repositori) i PDF	Markdown per a documentació viva al repo; PDF per a lliuraments tancats
Diagrames de xarxa	[!REVISAR: draw.io / Lucidchart / altre]	Diagrames d'arquitectura

Memòria final	Google Docs / Word	Format requerit pel mòdul
----------------------	--------------------	---------------------------

1.4 Planificació temporal

El mòdul de projecte abasta des del 16 de febrer de 2026 (incorporació d'ASO i I.Web) fins al 8 de maig de 2026 (entrega de la memòria), aproximadament 12 setmanes lectives. Les 198 hores totals es reparteixen en 5 sprints, un per miniprojecte. Tot i que els sprints són conceptualment seqüencials, en la pràctica s'han executat en paral·lel entre els dos membres de l'equip:

Calendari de sprints

Sprint	Període	Responsable	Miniprojecte	Hores
Sprint 1 Gestió de Projectes	16/02 – 08/05 (transversal)	Conjunt	Annex B – Gestió de Projectes	33
Sprint 2 Virtualització	16/02 – 22/03 (5 setmanes)	Axel	Proxmox bàsic + avançat	66
Sprint 3 Xarxa i Seguretat perimetral	16/02 – 15/03 (4 setmanes, paral·lel)	Daniel	Disseny xarxa + pfSense (transversal)	—
Sprint 4 IDS/IPS	16/03 – 05/04 (3 setmanes)	Daniel	Suricata	33
Sprint 5 Orquestració	23/03 – 12/04 (3 setmanes)	Axel	Kubernetes bàsic	33
Sprint 6 Monitoratge	06/04 – 26/04 (3 setmanes)	Daniel	Zabbix	33
Tancament Integració i memòria	27/04 – 08/05 (2 setmanes)	Conjunt	Proves cross-project, redacció memòria, preparació defensa	—

Tasques per sprint

A continuació es detallen les tasques principals identificades a Jira per a cada sprint. Els llistats complets, amb subtasques i estimacions, es troben al taulell del projecte.

Sprint Proxmox (Axel):

1. Instal·lació i configuració base de Proxmox VE sobre el servidor físic.
2. Configuració de xarxa al clúster i emmagatzematge.
3. Desplegament de plantilles i màquines virtuals base.
4. Configuració avançada: optimització de recursos i rendiment.

Sprint Xarxa (Daniel):

1. Disseny de la segmentació LAN / DMZ.
2. Instal·lació i configuració de pfSense.
3. Configuració de DDNS i registres CNAME a Nominalia.
4. Desplegament d'OpenVPN amb certificats.
5. Configuració de reverse proxy.

Sprint Suricata (Daniel):

1. Instal·lació de Suricata en mode IDS.
2. Configuració de regles i fonts d'amenaces.
3. Proves d'atac controlades i validació d'alertes.

Sprint Kubernetes (Axel):

1. Desplegament inicial amb Docker Compose com a referència.
2. Comparació amb Docker Swarm.
3. Implementació de clúster Kubernetes amb desplegament de microserveis.

Sprint Zabbix (Daniel):

1. Instal·lació del servidor Zabbix.
2. Desplegament d'agents a les màquines monitoritzades.
3. Configuració de dashboards i alertes.

1.5 Riscos identificats

A l'inici del projecte vam identificar els següents riscos potencials, amb el seu corresponent pla de mitigació:

Risc	Probabilitat	Impacte	Mitigació
Limitacions de hardware del servidor físic	Mitjana	Alt	Dimensionar VMs amb recursos mínims; apagar serveis no actius
Bloqueig de ports per part del proveïdor d'Internet (CGNAT)	Mitjana	Alt	Verificar IP pública real des del primer dia; alternativa amb túnel si cal
Caigudes de connexió domèstica	Baixa	Mitjà	Ús d'OpenVPN amb reconexió automàtica; treball local en cas de tall
Incompatibilitats entre miniprojectes en integrar-los	Mitjana	Mitjà	Definir interfícies de xarxa i reverse proxy de manera consensuada des del principi
Desviacions de calendari per dependències entre tasques	Alta	Mitjà	Treballar en paral·lel sempre que sigui possible; revisions setmanals

Pèrdua de dades de configuració	Baixa	Alt	Còpies de seguretat regulars al repositori GitHub; snapshots de Proxmox
Disponibilitat desigual dels membres	Baixa	Mitjà	Documentació al repo perquè qualsevol pugui continuar la feina de l'altre

1.6 Seguiment, desviacions i adaptacions

El seguiment s'ha realitzat de manera contínua a través del taulell de Jira, amb revisions informals al final de cada sprint i revisions setmanals presencials a l'aula. Tot i que la planificació general s'ha mantingut, hi ha hagut dues desviacions significatives respecte al pla inicial:

1.7 Retrospectives

Les retrospectives s'han realitzat al final de cada sprint de manera informal, sense documentació estructurada formal però amb conversacions explícites entre els dos membres de l'equip. Les principals lliçons extretes són:

- Sprint (Proxmox): la importància d'un dimensionament conservador inicial dels recursos. Les primeres VMs es van crear sobredimensionades i van caldre ajustaments.
- Sprint (Xarxa): la complexitat del DDNS amb el proveïdor d'Internet i la necessitat de validar el funcionament des de fora de la xarxa des del primer moment.
- Sprint (Suricata): la corba d'aprenentatge en l'escriptura de regles personalitzades; cal recolzar-se molt en conjunts de regles públiques (Emerging Threats).
- Sprint (Kubernetes): el networking de pods amb CNI és el punt més fràgil; cal triar la CNI adequada des de l'inici.
- Sprint (Zabbix): instal·lar agents en dispositius com pfSense requereix paquets específics que no sempre estan disponibles als repositoris oficials.

Si tornéssim a començar el projecte, documentaríem les retrospectives de manera formal des del primer sprint. Aquesta és una de les principals millores de gestió que aplicaríem.

Projecte Proxmox

1. Introducció i justificació tècnica

1.1 Objectiu del miniprojecte

Dissenyar i desplegar una infraestructura de virtualització professional basada en Proxmox VE que serveixi de base per a tots els serveis del projecte Hefesto (servidor web, Zabbix, Suricata, Kubernetes). En la fase bàsica (33 h) es construeix l'entorn de tres nodes amb emmagatzematge centralitzat. En la fase avançada (33 h) s'incorporen clúster, alta disponibilitat amb Ceph, replicació ZFS, backups automàtics i autoescalat de recursos.

1.2 Comparativa de plataformes i tria

Es van valorar quatre plataformes: VMware ESXi (referència empresarial però llicències costoses), Microsoft Hyper-V (excel·lent integració amb Windows Server però dependència ecosistema propietari), IsardVDI (open source orientat a VDI però limitat per a infraestructures complexes) i Proxmox VE (open source basat en Linux, KVM + LXC unificats, característiques empresarials sense cost de llicència).

Plataforma escollida: Proxmox VE 9.1. Combina control tècnic profund, codi obert i suport natiu per a clusterització, Ceph, ZFS i HA. La corba d'aprenentatge més pronunciada es compensa àmpliament amb la potència de l'eina i és, a més, una bona oportunitat formativa.

1.3 Repartiment de feina

Aquest miniprojecte ha estat liderat per Axel David Rodriguez com a responsable de la capa de virtualització. Daniel Pañero ha col·laborat en les decisions de xarxa i ha integrat la solució amb el pfSense, que es desenvolupa al bloc d'infraestructura comuna del projecte global.

2. Disseny de la solució

2.1 Arquitectura general

La infraestructura es construeix sobre VirtualBox amb virtualització anidada (Nested VT-x): tres nodes Proxmox, un NAS amb TrueNAS Core i un firewall pfSense, tots executant-se com a VMs sobre l'equip amfitrió. Aquesta opció permet reproduir un centre de dades complet amb un únic ordinador, facilita la portabilitat del laboratori i evita el cost d'un servidor físic dedicat.

Detall complet al repositori: Justificació tècnica completa de la virtualització anidada i flag VBoxManage → github.com/dpanero/Projecte_Hefesto_DPJ_ADR

2.2 Topologia de xarxa

Zona	Subxarxa	Funció
LAN	172.16.0.0/24	Gestió interna: nodes Proxmox, web TrueNAS, LAN del pfSense.
DMZ	Bridge sense IP al host	Serveis exposables. El propi Proxmox no té IP a la zona per minimitzar superfície d'atac.
NFS	172.17.0.0/24	Tràfic d'emmagatzematge dedicat per evitar saturar la LAN durant migracions/backups.

2.3 Decisions clau

- **Tres nodes:** mínim per a quòrum del clúster i per a Ceph amb Size=3, Min Size=2.
- **Storage centralitzat NFS + Ceph:** NFS per a ISOs i backups (senzill); Ceph per als discos virtuals d'HA (distribuït i tolerant).
- **DMZ sense IP a l'hipervisor:** una VM compromesa a la DMZ no pot atacar Proxmox directament.
- **Plantilla Ubuntu + cloud-init:** totes les VMs del projecte deriven de la mateixa base, garantint coherència i desplegament ràpid.

3. Implementació de la fase bàsica (33 h)

3.1 Preparació de l'entorn (VirtualBox)

Cada node Proxmox és una VM de VirtualBox amb 8192 MB de RAM, 10 vCPUs, EFI, paravirtualització KVM i — peça clau — el flag Nested VT-x/AMD-V activat via VBoxManage. Cada VM disposa de tres discos (80 GB sistema, 30 GB Ceph, 30 GB ZFS) i tres adaptadors de xarxa interna (LAN, DMZ, NFS) en lloc de bridge per garantir aïllament del host.

Detall complet al repositori: Configuració completa pas a pas de la VM contenidora → github.com/dpanero/Projecte_Hefesto_DPJ_ADR

3.2 Instal·lació de Proxmox VE 9.1

Procés de l'instal·lador gràfic estàndard: acceptació de l'EULA, selecció del disc primari de 80 GB amb ext4 (els altres dos discos queden lliures per a Ceph i ZFS), configuració regional Spain/Europe-Madrid, contrasenya robusta de root i correu d'alertes. Configuració de xarxa de gestió:

Paràmetre	Valor
Hostname (FQDN)	proxmox.hefesto.local (i proxmox2/3 per als altres nodes)
IP / CIDR	172.16.0.10/24 · 172.16.0.8/24 · 172.16.0.6/24
Gateway	172.16.0.1 (pfSense)
Interface de gestió	nic0

El procés es repeteix idènticament als tres nodes canviant hostname i IP. Després del primer arrencada, cada node és accessible per HTTPS al port 8006.

Detall complet al repositori: Manual d'instal·lació detallat amb captures →

github.com/dpanero/Projecte_Hefesto_DPJ_ADR

3.3 Configuració de xarxa al node

A Proxmox la connectivitat es construeix sobre Linux Bridges (switches virtuals). S'han creat tres bridges per node, alineats amb les zones de xarxa:

Bridge	IP al host	Pont físic	Observació
LAN	172.16.0.10/24	nic0	Bridge principal de gestió i de la majoria de VMs.
DMZ	Sense IP	enp0s8	Switch transparent. Decisió deliberada per seguretat.
NFS	172.17.0.3/24	enp0s9	Tràfic d'emmagatzematge aïllat de la LAN.

El pfSense actua com a passarel·la entre zones, fa NAT cap a Internet i hostatja l'OpenVPN. La seva configuració detallada es desenvolupa al bloc d'infraestructura comuna del projecte global.

3.4 Emmagatzematge centralitzat amb TrueNAS Core

Es desplega un NAS amb TrueNAS Core (FreeBSD + ZFS, gratuït) com a VM de VirtualBox amb dues NICs: una a la LAN (172.16.0.9, gestió web) i un Bond FAILOVER de dues interfícies a la xarxa NFS (172.17.0.2, tràfic de discos). El protocol FAILOVER s'escull en lloc de LACP perquè el switch virtual de VirtualBox no suporta agregació real.

Pool ZFS poolvms: RAIDZ1 de 5 discos de 40 GB (tolera la fallada d'un disc) + Log VDEV (4 GB, accelera escriptures síncrones) + Cache VDEV (16 GB) + Spare (40 GB) + Dedup (8 GB).

Sobre el pool, un dataset dedicat **dataset_NFS** exposat per NFS només als hosts 172.17.0.3-5 (els tres nodes Proxmox), amb Maproot User=root perquè Proxmox pugui crear correctament els fitxers de discos virtuals.

Per què NFS i no iSCSI o SMB: NFS és el protocol estàndard de facto per a virtualització Linux, més senzill que iSCSI i molt més eficient que SMB per a discos virtuals.

Integració a Proxmox des de Datacenter → Storage → Add → NFS amb ID TrueNAS_Hefesto, content type Disk image + ISO + Container template + VZDump.
Disponible immediatament als tres nodes.

Detall complet al repositori: Configuració completa del NAS, pool ZFS, share NFS i integració → github.com/dpanero/Projecte_Hefesto_DPJ_ADR

3.5 Plantilla d'Ubuntu Server amb cloud-init

Per disposar d'una base reproduïble es crea una VM única amb Ubuntu Server 24.04 LTS (VM ID 100) amb VirtIO SCSI Single, QEMU Agent activat, disc qcow2 amb thin provisioning, CPU x86-64-v2-AES (acceleració hardware AES) i Ballooning Device habilitat. Es converteix a plantilla després d'esborrar tots els identificadors únics (machine-id, claus SSH del servidor, configuració de xarxa) i instal·lar el paquet cloud-init.

Cloud-init: estàndard de la indústria (AWS, Azure, OpenStack) per injectar configuració al primer arrencada del clon. Cada VM clonada hereta usuari, contrasenya, claus SSH i configuració d'IP per DHCP a partir de la configuració definida a la plantilla.

Validació: clonant la plantilla amb Mode = Full Clone, la nova VM agafa una IP DHCP nova, accepta SSH amb les claus configurades i té noves claus de servidor SSH. Procés repetible en segons.

Detall complet al repositori: Procediment complet de creació de la plantilla i clonació → github.com/dpanero/Projecte_Hefesto_DPJ_ADR

3.6 Perfils de recursos i optimització

S'han definit tres perfils de recursos en funció del rol de cada VM:

Perfil	VM exemple	vCPU	RAM	Justificació
Frontend DMZ	web (102)	3	2 GB	Pic curt; delega pes a la BD.
Monitoratge	Zabbix (995)	2	2 GB	Càrrega lleugera però constant.
IA / Anàlisi	hefesto-ai (999)	2	3 GB	Memòria intensiva, CPU pic puntual.

A totes les VMs s'activen dos optimitzadors clau de RAM: Ballooning Device (Proxmox reclama RAM no usada per la VM i la reassigna a altres VMs que sí la necessiten) i KSM (Kernel Same-page Merging — fusiona pàgines de memòria idèntiques entre VMs, especialment efectiu quan totes deriven de la mateixa plantilla). El hotplug de CPU/RAM/disc es deixa preconfigurat però s'aprofundeix a la fase avançada.

Detall complet al repositori: Detall dels perfils, ballooning, KSM i hotplug bàsic → github.com/dpanero/Projecte_Hefesto_DPJ_ADR

4. Implementació de la fase avançada (33 h)

4.1 Clusterització dels tres nodes

Des de Datacenter → Cluster del primer node es crea ClusterHefesto amb la xarxa de Corosync sobre la LAN. Les altres dues màquines s'uneixen via Join Information (cadena codificada amb empremta digital del mestre). Els tres nodes apareixen amb Quorum: Yes, indicant que el clúster és plenament operatiu i centralitzat: una sola interfície web, mètriques agregades, polítiques globals.

4.2 Servidor de backups (Proxmox Backup Server)

S'instal·la una quarta VM amb Proxmox Backup Server, connectada a la LAN (administració) i a la xarxa NFS (tràfic de backups, per no saturar la LAN). Es crea un datastore al disc local del PBS, s'obté el fingerprint sha256 amb proxmox-backup-manager cert info i es registra el PBS al clúster Proxmox com a storage. Tasques de backup programades en mode snapshot a les 21:00 cada dia. Validació amb un backup forçat i posterior restauració a un altre node — la nova VM apareix idèntica a l'original.

Detall complet al repositori: Configuració completa del PBS, registre al clúster i validació de restauració → github.com/dpanero/Projecte_Hefesto_DPJ_ADR

4.3 Alta disponibilitat amb Ceph

Ceph és el sistema d'emmagatzematge distribuït que sustenta l'HA. Converteix els discos /dev/sdb dels tres nodes en un únic pool replicat. Tres components clau:

- **OSD (Object Storage Daemon):** un per node sobre /dev/sdb, formatat amb BlueStore.
- **Monitor (MON):** un per node (nombre senar per garantir quòrum i evitar split-brain).
- **Manager (MGR):** inicialment només al primer node, però se n'afegeixen als altres dos perquè la dashboard de Ceph també sigui d'alta disponibilitat.

Pool HefestoPool1 amb Size=3 (cada bloc replicat 3 còpies, un per node), Min Size=2 (el clúster opera amb 2 nodes vius), 128 PGs (recomanat per a 3 OSDs). Marcat com Add as Storage perquè aparegui automàticament als tres nodes com a destinació RBD.

Configuració del gestor d'HA: a Datacenter → HA → Resources, s'afegeixen les VMs crítiques amb Max. Restart=1, Max. Relocate=2 i Failback=true. La política de shutdown és migrate, perquè en apagar un node ordenadament les VMs es desplacin automàticament. Davant la caiguda d'un node, el clúster aïlla el servidor (fencing) i ordena als nodes supervivents que arrenquin les VMs en qüestió de segons.

4.4 Replicació ZFS (alternativa lleugera a Ceph)

Per a entorns amb menys ample de banda o discos disponibles, Proxmox ofereix replicació ZFS asíncrona. A diferència de Ceph (sincrònica), ZFS envia snapshots incrementals a un node destí cada cert temps; més lleugera, però amb RPO > 0 (es poden perdre canvis posteriors a l'últim snapshot).

Es crea un pool ZFS-Replication a /dev/sdc de cada node (amb el mateix nom als tres — requisit indispensable) i es declara com a storage al Datacenter. Per a la VM 992 ubicada al pool ZFS, s'afegeix un Replication Job amb destí proxmox3 i schedule */2 (en producció seria 15-30 min). El primer cicle envia el snapshot complet (~186 MB), els següents són només incrementals. El log mostra freeze guest filesystem → snapshot → send/recv → thaw, garantint snapshots consistents gràcies al QEMU Guest Agent.

Detall complet al repositori: Detall complet de Ceph (instal·lació, OSDs, pools, MONs, MGRs) i ZFS Replication → github.com/dpanero/Projecte_Hefesto_DPJ_ADR

4.5 Elasticitat: Hotplug + autoescalat

Proxmox no escala recursos automàticament, però ho he resolt combinant tres peces:

- **Hotplug habilitat (Disk, Network, USB, Memory, CPU):** permet afegir recursos sense reiniciar la VM.
- **Hardware preparat per créixer:** Cores màxim > vCPUs actives, RAM màxima > mínima, NUMA habilitat (requisit per a hotplug de RAM), Ballooning Device i QEMU Guest Agent actius.
- **Script /root/autoescala.sh + cron:** llegeix l'ús de CPU amb `qm status` i, si supera el 80%, afegeix un core en calent. Programat cada minut amb cron, log a `/var/log/autoescala.log`.

Validació amb càrrega artificial: en menys d'un minut l'script detecta CPU > 80% i amplia les vCPUs sense reiniciar, deixant constància al log amb el missatge "Alta carga detectada... Aumentando CPU a NEW_CORES en caliente".

Detall complet al repositori: Codi de l'script d'autoescalat i procediment de validació → github.com/dpanero/Projecte_Hefesto_DPJ_ADR

4.6 Monitoratge integrat (clúster · node · VM)

Proxmox ofereix monitoratge a tres nivells. Aquesta jerarquia és clau per diagnosticar problemes ràpidament: si un símptoma apareix a nivell de clúster afecta tothom; si només es veu a una VM, el problema està localitzat.

Nivell	Mètriques principals	Quan baixar a aquest nivell
Clúster	Health, Quorum, ús agregat CPU/RAM/Storage, estat Ceph (HEALTH_OK)	Punt d'entrada. Si tot és verd, els problemes són locals.
Node	CPU/Load Average, Memory (Used/Cache/ARC), IO Pressure Stall, Network Traffic, S.M.A.R.T.	Quan un símptoma apunta a un servidor concret.
VM	CPU/RAM realment usats (gràcies al Ballooning), Disk IO, Network	Per ajustar recursos i detectar sobre/subprovisionament.

Limitació: l'eina integrada no guarda històric a llarg termini ni permet alertes proactives. Per això el projecte global complementa el monitoratge amb un Zabbix dedicat (miniprojecte 5).

5. Validació i proves

Cinc proves clau executades per validar la infraestructura:

Prova	Procediment	Resultat
1. Migració en viu	VM 996 amb ping continu, Migrate → Online cap a un altre node.	Sense desconexió SSH ni pèrdua de paquets. Lleu pic puntual de latència.
2. Replicació ZFS	VM 992 al pool ZFS, escriure dades, esperar 2 min, comprovar al destí.	Primer cicle complet (~186 MB), següents només incrementals. Snapshots consistents.
3. Backup + Restore (PBS)	Forçar backup, esborrar VM, restaurar a un altre node.	Nova VM idèntica a l'original sense pèrdua de dades.
4. Autoescalat de CPU	stress --cpu 4 --timeout 120 dins la VM amb hotplug actiu.	En <1 min l'script detecta >80% i amplia vCPUs en calent. Log registra l'esdeveniment.
5. Tolerància a fallada (HA)	Apagat brusc del node proxmox amb VMs HA assignades.	En ~60 s el clúster aïlla el node i arrenca les VMs als supervivents. Servei recuperat en 1-2 min.

Detall complet al repositori: Captures i logs de cada prova al detall →

github.com/dpanero/Projecte_Hefesto_DPJ_ADR

6. Conclusions i reflexió

6.1 Objectius assolits

Tots els objectius plantejats s'han assolit. La fase bàsica ha proporcionat una infraestructura virtual operativa amb tres nodes, emmagatzematge centralitzat i plantilles cloud-init. La fase avançada ha portat la solució a nivell de producció amb clúster, Ceph, HA, replicació ZFS, backups automàtics amb PBS i autoescalat. Cobrim els tres pilars d'un entorn modern:

- **Disponibilitat:** sense punts únics de fallada (clúster + Ceph + HA).
- **Escalabilitat:** recursos elàstics en calent (Hotplug + autoescalat).
- **Observabilitat:** monitoratge a tres nivells (clúster, node, VM).

6.2 Dificultats i com s'han resolt

- **Virtualització anidada:** calia activar Nested VT-x via VBoxManage des del host abans d'arrencar Proxmox; sense aquest pas, KVM no funcionava amb acceleració hardware.
- **Bond LACP impossible:** el switch virtual de VirtualBox no suporta agregació real, així que es va canviar a FAILOVER, més adequat al laboratori.
- **Replicació ZFS amb noms diferents:** el primer intent va fallar perquè els pools dels tres nodes tenien noms diferents. Un cop unificats com ZFS-Replication, va funcionar a la primera.
- **Manager Ceph únic:** per defecte només se'n crea un al primer node. Es van afegir Managers addicionals als altres dos perquè la dashboard també fos d'alta disponibilitat.

6.3 Possibles millores futures

- **Migració a hardware físic dedicat:** el laboratori sobre VirtualBox té sostre de rendiment per la doble capa de virtualització.
- **Infraestructura com a codi:** automatitzar el desplegament amb Terraform o Ansible (actualment tot via interfície web).
- **Backups encriptats fora del laboratori:** el PBS desa als seus discos locals; en producció haurien de viatjar a un emplaçament físic separat.
- **Integració amb Zabbix per a alertes proactives:** complementar el monitoratge integrat de Proxmox amb el Zabbix del projecte global.
- **Proves de ruptura més agressives:** simular caigudes simultànies de més d'un node, talls de xarxa o fallades de discos físics per validar el límit de la tolerància.

6.4 Reflexió personal

Aquest miniprojecte ha estat el més tècnicament exigent del projecte Hefesto en termes d'amplitud: tocar des d'instal·lació d'un hipervisor fins a HA amb storage distribuït en 66 hores ha requerit una planificació molt acurada. La principal lliçó és la importància d'una bona base: si la fase bàsica (xarxa, emmagatzematge, plantilles) està ben dissenyada, la fase avançada flueix gairebé sola. En canvi, qualsevol decisió presa al principi acaba pagant-se cara quan es vol activar funcionalitats com la migració en viu o l'autoescalat.

Treballar amb Proxmox m'ha demostrat per què és l'estàndard de virtualització open source: la profunditat tècnica que ofereix és comparable a solucions empresarials com VMware,

però amb el control total propi del programari lliure. És, sense cap dubte, una competència professionalment molt valorable.

Projecte Docker-Orquestradors

1. Introducció i arquitectura general

ShopMicro és una plataforma fictícia de comerç electrònic que simula la migració d'una aplicació monolítica a una arquitectura de microserveis containeritzats. El projecte progressa per quatre fases de complexitat creixent: desenvolupament local amb Docker Compose, alta disponibilitat amb Docker Swarm, aplicació de seguretat en profunditat i, finalment, migració a Kubernetes.

1.1 Serveis de la plataforma

Servei	Tecnologia	Funció
frontend	Nginx + HTML/JS	Interfície web estàtica
api-gateway	Nginx	Reverse proxy, punt d'entrada únic
product-service	Python Flask	CRUD productes, cache Redis (TTL 60s)
order-service	Python Flask	Gestió comandes + publicació RabbitMQ
user-service	Python Flask	Autenticació JWT
notification-service	Python Flask	Consumidor de la cua RabbitMQ
db-products	MySQL 8.0	Base de dades de productes
db-orders	MySQL 8.0	Base de dades de comandes i usuaris
cache	Redis 7	Cache amb TTL 60 s
message-queue	RabbitMQ 3	Cua de missatges asíncrona

1.2 Fluxos funcionals (verificats a totes les fases)

Flux 1 — Consulta de productes: frontend → api-gateway → product-service → Redis (cache) o db-products.

Flux 2 — Creació de comanda: order-service descompta stock, crea comanda i publica missatge → notification-service registra al log.

Flux 3 — Tolerància a fallades: parada manual d'un node i comprovació de redistribució de rèpliques.

1.3 Tecnologies

Àmbit	Tecnologies
Containerització	Docker Engine 24+, Docker Compose v2, Docker Swarm
Microserveis	Python 3.11, Flask, PyJWT, pika (RabbitMQ), redis-py
Infraestructura	Ubuntu Server 24, VirtualBox (3 VMs), WSL2
Registry	Docker Hub (arodriguez5)
Seguretat	Docker Secrets, mTLS automàtic Swarm, Docker Scout
Orquestració avançada	Kubernetes 1.30, Minikube, kubect!, kompose

2. Docker Compose: Entorn de desenvolupament

Objectiu: posar en marxa els 10 serveis en un entorn local amb Docker Compose v2, aplicant xarxes internes separades, volums persistents i healthchecks.

2.1 Arquitectura de xarxes

S'han creat tres xarxes bridge per aïllar els serveis per responsabilitat:

Xarxa	Serveis connectats	Motiu
frontend-net	frontend, api-gateway	Separa la capa de presentació
backend-net	api-gateway, product, order, user, notification	Aïlla els microserveis
data-net	microserveis, db-products, db-orders, cache, mq	Protegeix les dades

Únicament el frontend exposa el port 8080 a l'exterior. Les bases de dades no publiquen cap port.

2.2 Elements clau del docker-compose.yml

- image: / build: — imatges de Docker Hub o construïdes localment amb Dockerfile.
- depends_on amb condition: service_healthy — els microserveis esperen que les BD i Redis estiguin healthy.
- healthcheck — definit a les BD (mysqladmin ping), Redis (redis-cli ping) i RabbitMQ.
- volumes — db_products_data i db_orders_data mantenen les dades entre reinicis.
- Secrets en fitxers .txt locals (migrats a Docker Secrets a la Fase 3).

2.3 Patrons implementats als microserveis

Patró	Servei	Descripció
Cache-aside	product-service	Consulta Redis primer; si no hi ha dada, va a MySQL i l'emmagatzema (TTL 60s). El camp source indica l'origen.
Messaging	order-service → notif	En crear comanda: descompta stock, insereix a BD i publica missatge a RabbitMQ. notification-service el consumeix.
JWT Auth	user-service	Registro, login i generació de token JWT. Contrasenyes hashejades amb SHA-256.

2.4 Resultats i aprenentatges

- Tots els 9 serveis containeritzats i els 3 fluxos funcionals verificats des del navegador.
- Comprensió de la diferència entre image: (Docker Hub) i build: (Dockerfile local).
- Importància dels healthchecks per evitar race conditions d'arrencada.
- Patró cache-aside amb Redis i missatgeria asíncrona amb RabbitMQ.

[Documentació completa Fase 1 \(docker-compose.yml, codi microserveis, captures\)](#)

3. Docker Swarm: Alta disponibilitat

Objectiu: convertir l'entorn de la Fase 1 en un clúster Docker Swarm de 3 nodes (1 manager + 2 workers) amb rèpliques, restart policies i rolling updates.

3.1 Infraestructura

Node	Rol	IP interna	IP host-only
manager	Manager Swarm	192.168.0.1	192.168.56.2
worker1	Worker Swarm	192.168.0.2	192.168.56.3
worker2	Worker Swarm	192.168.0.3	192.168.56.4

Cada VM té dos adaptadors de xarxa: xarxa interna per a la comunicació entre nodes i xarxa host-only per a la connexió SSH des de Windows.

3.2 Diferències Compose → Swarm (docker-stack.yml)

Concepte	Compose (Fase 1)	Swarm (Fase 2)
build:	Construeix localment	No suportat — cal image: a Docker Hub
depends_on	Espera healthchecks	Ignorat — cal retry logic al codi
deploy:	Ignorat	Configuració principal de Swarm
Xarxes	bridge	overlay (multi-node)
Scaling	Manual	docker service scale

3.3 Configuració de deploy: aplicada

- replicas: 2 per als microserveis d'API; replicas: 1 per a les BD.
- placement.constraints: node.role == manager per a BD i cua (consistència de volums).
- restart_policy: on-failure amb delay 5s.
- update_config: parallelism 1, order start-first → rolling update sense downtime.

3.4 Race condition detectada i solució

En el primer desplegament, els microserveis no podien inicialitzar les taules perquè MySQL no havia completat la creació de l'esquema. En un entorn multi-node amb xarxes overlay, el procés d'inici és més lent que en Compose.

Solució: incrementar els intents de init_db() de 10 a 30 i l'interval de 3 a 5 s (finestra de 150 s). Les imatges es van reconstruir com a :1.1 i es van actualitzar al clúster amb docker service update --force.

3.5 Demostracions realitzades

Demostració	Comanda	Resultat
Routing mesh	curl http://192.168.0.{1,2,3}:8080/api/products	El port 8080 respon des de qualsevol node
Tolerància a fallades	sudo systemctl stop docker (worker2)	Swarm redistribueix rèpliques al manager i worker1 en ~30 s
Escalat en calent	docker service scale shopmicro_product-service=4	De 2 a 4 rèpliques sense interrupció del servei

[Documentació completa Fase 2 \(docker-stack.yml, captures, comandes\)](#)

4. Seguretat al clúster Swarm

Objectiu: migrar ShopMicro de Docker Swarm a Kubernetes amb Minikube (K8s v1.30), reproduint tota la funcionalitat de les fases anteriors amb els conceptes propis de Kubernetes.

4.1 Vulnerabilitats identificades a la Fase 2

ID	Vulnerabilitat	Servei afectat	Risc
V1	Contrasenya MySQL apppass123 en clar a DB_PASSWORD	product, order, user	Alt
V2	Credencials RabbitMQ admin/admin123 en clar	message-queue, order, notification	Mitjà-Alt
V3	Clau JWT clausecreta_2asix en clar	user-service	Alt
V4	Usuari de BD appuser visible a docker inspect i als logs	Tots els microserveis	Baix

4.2 Docker Secrets

Els Docker Secrets s'emmagatzemen xifrats en repòs al Raft del clúster, viatgen xifrats entre nodes (mTLS) i es munten com a fitxers a /run/secrets/<nom> — mai com a variables d'entorn. S'han creat 5 secrets:

Secret	Valor substituït	Microserveis amb accés
db_root_password	Contrasenya root MySQL	db-products, db-orders
db_user_password	Contrasenya appuser	product, order, user
mq_user	Usuari RabbitMQ	order, notification
mq_password	Contrasenya RabbitMQ	order, notification
jwt_secret	Clau de signatura JWT	user-service

S'ha afegit una funció `read_secret()` a cada microservei Python amb fallback a variable d'entorn per compatibilitat amb Compose en local.

4.3 Cas especial: RabbitMQ 3.13+

La imatge `rabbitmq:3-management` va eliminar el suport a `RABBITMQ_DEFAULT_USER_FILE`. S'ha migrat a un fitxer de definicions JSON (carregueu per `load_definitions` al `rabbitmq.conf`) que conté el hash SHA-256 de la contrasenya i es munta com a secret.

4.4 Aïllament de xarxes overlay

La Fase 2 tenia 3 xarxes obertes entre serveis. A la Fase 3 s'han creat 6 xarxes especialitzades seguint el principi de mínim privilegi:

Xarxa	Serveis connectats
frontend-net	frontend ↔ api-gateway
backend-net	api-gateway ↔ product, order, user
products-net	product-service ↔ db-products, cache
orders-net	order-service, user-service ↔ db-orders
messaging-net	order-service, notification-service ↔ message-queue
notif-net	notification-service ↔ message-queue (read-only)

Resultat: si un atacant compromet `notification-service`, no pot accedir a cap base de dades.

4.5 TLS automàtic de Swarm

Docker Swarm inclou una CA interna que emet certificats per a cada node en el moment de join. Tots els missatges de control entre nodes viatgen amb mTLS. Els certificats es renoven automàticament cada 3 mesos (configurable). Aquest mecanisme és transparent i no requereix configuració addicional.

4.6 Escaneig de vulnerabilitats (Docker Scout)

Imatge	Crítiques	Altes	Total
mysql:8.0	1	12	32
redis:7-alpine	5	45	91
rabbitmq:3-management	3	15	85
nginx:alpine	0	2	15
arodriguez5/shopmicro-product-service	0	5	5
arodriguez5/shopmicro-order-service	0	9	33
arodriguez5/shopmicro-user-service	0	10	34
arodriguez5/shopmicro-notification-svc	0	5	29
arodriguez5/shopmicro-frontend	0	2	15
arodriguez5/shopmicro-api-gateway	0	2	15
TOTAL	9	107	~353

CVEs destacades

- CVE-2025-68121 (CRITICAL) — stdlib Go v1.24.6 a mysql:8.0. Afecta l'entrypoint, no el motor MySQL. Risc real baix: contenidors no exposats a Internet.
- CVE-2024-21272 (HIGH) — mysql-connector-python < 9.1.0. SQL Injection al connector. Mitigació: actualitzar requirements.txt a 9.1.0 → imatges :1.2.
- CVE-2026-24049 (HIGH) — wheel v0.45.1. Path traversal. Risc real molt baix: eina usada només durant el build, no en runtime.

4.7 Defensa en profunditat

L'objectiu no és zero CVEs (pràcticament impossible), sinó que cap CVE individual sigui suficient per comprometre el sistema. Les mesures aplicades cobreixen: aïllament de xarxes, secrets muntats com a fitxers, constraints de placement per a BD, i mTLS automàtic entre nodes.

[Documentació completa Fase 3 \(docker-stack.yml segur, captures Docker Scout\)](#)

5. Kubernetes: Entorn de producció

Objectiu: migrar ShopMicro de Docker Swarm a Kubernetes amb Minikube (K8s v1.30), reproduint tota la funcionalitat de les fases anteriors amb els conceptes propis de Kubernetes.

5.1 Swarm vs Kubernetes — Comparativa clau

Aspecte	Docker Swarm (Fases 2-3)	Kubernetes (Fase 4)
Definició	1 fitxer docker-stack.yml	10+ manifests YAML separats
Escalat automàtic	No nadiu	HorizontalPodAutoscaler (HPA)
Health checks	healthcheck: simple	livenessProbe + readinessProbe
Rolling updates	update_config (parallelism, order)	RollingUpdate (maxSurge, maxUnavailable)
Rollback	Versió anterior únicament	kubectl rollout history (qualsevol revisió)
Secrets	Xifrats en repòs (Raft)	Base64 — cal Sealed Secrets o Vault
Networking	Routing mesh automàtic	Services: ClusterIP / NodePort / LoadBalancer
Persistència	Volums Docker al node	PersistentVolumes + PVCs + StorageClasses
Complexitat	Baixa (5 comandes essencials)	Alta (desenes de recursos i comandes)

5.2 Estructura de manifests Kubernetes

- k8s/namespace/ — Namespace shopmicro per aïllar tots els recursos.
- k8s/secrets/ — Kubernetes Secrets (Base64) per a credencials.
- k8s/configmaps/ — ConfigMaps per a configuració no sensible.
- k8s/generated/ — Deployments + PVCs generats amb kompose convert i adaptats manualment.
- k8s/services/ — Services ClusterIP per a comunicació interna + LoadBalancer per al frontend.

5.3 Init containers

A diferència de depends_on (ignorat per Swarm) o del retry loop de la Fase 2, a Kubernetes s'han afegit init containers que executen una comprovació fins que MySQL accepta

connexions. Quan l'init container acaba correctament, el contenidor principal arrencarà amb la BD llesta.

5.4 Rolling update demostrat

Es va actualitzar product-service de :1.1 a :1.2 amb maxSurge: 1 i maxUnavailable: 0.

Kubernetes va seguir el procés: crear pod nou → esperar readinessProbe → eliminar pod antic. En cap moment hi va haver menys de 2 pods disponibles → zero downtime.

Kubectl rollout history permet consultar l'historial de revisions i fer rollback a qualsevol versió anterior, funcionalitat que Swarm no ofereix.

5.5 Fluxos funcionals verificats

Flux	Verificació
Consulta productes + Redis	Primera petició → source: db; segona dins 60s → source: cache. Confirmat als logs del pod.
Creació comanda + RabbitMQ	Missatge 'Comanda creada' al frontend + log [NOTIFICACIÓ] al pod notification-service.
Self-healing	kubectl delete pod <pod> → Kubernetes crea un nou en <15s. L'altra rèplica continua servint.

5.6 Conclusions de la Fase 4

- Probes granulars: detecten si el servei respon realment (no només si el procés viu).
- Històric de versions i rollback a qualsevol revisió anterior.
- Init containers: solució neta al problema de dependències entre serveis.
- Secrets en Base64 (no xifrats per defecte): limitació respecte als Docker Secrets de Swarm.
- Complexitat operativa molt més alta: 30+ fitxers enfront d'un sol docker-stack.yml.

[Documentació completa Fase 4 \(manifests K8s, captures, comparativa Swarm-K8s\)](#)

6. Conclusions globals

El projecte ShopMicro ha permès recórrer el cicle complet d'una migració real: de l'entorn de desenvolupament local fins a la producció amb Kubernetes, passant per l'alta disponibilitat i la seguretat en profunditat.

6.1 Evolució de l'arquitectura

Fase	Tecnologia	Valor aportat
1 — Compose	Docker Compose v2	Containerització de tots els serveis, healthchecks, cache-aside, missatgeria asíncrona.
2 — Swarm	Docker Swarm	Clúster multi-node, rèpliques, routing mesh, tolerància a fallades, escalat en calent.
3 — Seguretat	Swarm + Scout	Docker Secrets, aïllament de xarxes overlay, mTLS automàtic, escaneig CVEs.
4 — Kubernetes	K8s + Minikube	Probes granulars, historial de versions, init containers, HPA (escalat automàtic).

6.2 Conceptes apresos

- Diferència entre containerització (Docker), orquestració local (Compose) i orquestració de clúster (Swarm/Kubernetes).
- Principi de mínim privilegi aplicat a xarxes overlay i secrets.
- Gestió de race conditions entre serveis dependents en entorns distribuïts.
- TLS mutu automàtic de Swarm per a la comunicació segura entre nodes.
- Defensa en profunditat: les vulnerabilitats no s'eliminen, es mitiguen per capes.
- Diferències pràctiques entre Docker Secrets (xifrats en Raft) i Kubernetes Secrets (Base64).
- Avantatges de Kubernetes: HPA, rollout history, probes granulars, init containers.
- Quan triar Swarm (equips petits, baixa complexitat) vs Kubernetes (producció a escala, cloud).

6.3 Limitacions i millores futures

- CVEs no resoltes: caldria un cicle d'actualització mensual de les imatges i pinning per digest.
- Xifrat de xarxes overlay (--opt encrypted) no activat: necessari en cloud públic.
- Kubernetes Secrets en Base64: integrar Sealed Secrets o HashiCorp Vault per a xifrat real.

- CI/CD pipeline: integrar docker scout cves com a gate de qualitat abans de publicar imatges.
- Multi-stage builds i USER no-root als Dockerfiles per reduir superfície d'atac.

IDS/IPS amb Suricata

1. Objectiu del sistema IDS/IPS dins d'Hefesto

Resum final de la part Suricata, centrada en què s'ha fet, per què i què s'ha validat.

La part de Suricata del projecte Hefesto s'ha plantejat com una capa de seguretat afegida al firewall. El firewall ja controla què entra i què surt segons IP, ports i regles, però Suricata aporta inspecció del trànsit i detecció de patrons sospitosos que un filtratge normal no sempre veu.

L'objectiu no ha estat muntar un IDS/IPS de forma teòrica, sinó integrar-lo dins de la xarxa real del laboratori: WAN, LAN, DMZ, VPN, Proxmox, servidor web i Zabbix. Això permet que les alertes tinguin context i no quedin com missatges aïllats sense relació amb la infraestructura.

Suricata s'ha desplegat sobre pfSense perquè pfSense ja és el punt de pas principal entre xarxes. Aquesta decisió té sentit en un laboratori com el nostre, ja que evita muntar un servidor dedicat per IDS/IPS i aprofita una peça que ja estava al centre de la topologia.

Durant el projecte s'ha treballat la diferència entre detectar i bloquejar. En una primera fase és més prudent observar alertes, entendre quines són útils i reduir falsos positius. Després, quan una regla o una situació és clara, es pot aplicar una resposta més preventiva.

Detectar escanejos, intents d'accés i tràfic anòmal.

Aplicar prevenció només quan no trenqui el funcionament normal.

Separar polítiques segons WAN, LAN, DMZ i VPN.

Generar logs consultables des de pfSense i Zabbix.

Validar la solució amb proves controlades contra serveis de la DMZ.

El resultat final és una capa de visibilitat i resposta que complementa la segmentació de xarxa i millora la seguretat global d'Hefesto.

3. IDS, IPS i criteri aplicat al projecte

Un IDS és un sistema de detecció: observa el trànsit, compara amb regles o signatures i genera alertes. Un IPS va més enllà i pot bloquejar o tallar trànsit quan detecta un patró perillós. La diferència sembla simple, però a la pràctica és una de les decisions més delicades del projecte.

Si només fem IDS, tenim visibilitat però no aturem l'atac automàticament. Si activem IPS massa aviat, podem bloquejar trànsit legítim i deixar serveis sense funcionar. Per això s'ha treballat amb un criteri progressiu: primer detectar, després revisar alertes i finalment bloquejar només quan el comportament és clar.

A Hefesto això és especialment important perquè hi ha serveis reals a la DMZ, accés VPN, NAT, regles de firewall i proves de connexió des de fora. Un bloqueig massa agressiu pot semblar seguretat, però si deixa la web inaccessible o talla connexions legítimes, el resultat no és bo.

Mode	Avantatge	Risc
IDS	Permet estudiar alertes sense tallar servei	No bloqueja automàticament
IPS	Pot aturar trànsit sospitosos en temps real	Pot generar falsos bloquejos
Mixt	Detectar molt i bloquejar només allò clar	Requereix revisió i ajust

El criteri final ha estat entendre Suricata com una eina que ha de reforçar el firewall, no substituir-lo. pfSense continua decidint què està permès, i Suricata inspecciona el trànsit per trobar patrons que poden indicar reconeixement, explotació o activitat anòmla.

Aquesta forma de treball és més segura i més realista per un entorn de projecte.

4. Selecció de tecnologia: per què Suricata

Abans d'escollir Suricata s'han comparat diferents opcions IDS/IPS. Snort és una eina molt coneguda i històrica, Suricata és una alternativa moderna amb bon rendiment i OSSEC encaixa més com a HIDS, és a dir, monitorització des del punt de vista del host.

La necessitat del projecte era analitzar trànsit de xarxa en el perímetre i en zones internes. Per aquest motiu OSSEC no era l'opció principal, tot i que podria ser complementària en un futur per controlar fitxers, logs locals o integritat de servidors.

Entre Snort i Suricata, s'ha triat Suricata perquè encaixa molt bé amb pfSense, aprofita millor sistemes multi-thread i disposa de formats de log moderns com EVE JSON. També hi ha molta documentació i categories de regles útils per un laboratori amb DMZ i serveis web.

Eina	Tipus	Valoració dins del projecte
Snort	NIDS/NIPS	Molt conegut, però menys atractiu pel nostre enfocament
Suricata	NIDS/NIPS	Escollit per rendiment, integració i logs
OSSEC	HIDS	Interessant com a complement, no com a IDS de xarxa principal

També es va valorar si muntar Suricata en un servidor dedicat o dins de pfSense. En un entorn de molt trànsit podria tenir sentit separar-lo, però en Hefesto pfSense és suficient i redueix complexitat. A més, així firewall i IDS/IPS queden administrats des del mateix punt.

La decisió final és pràctica: Suricata sobre pfSense dona prou potència i és coherent amb la mida del laboratori.

5. pfSense com a base de firewall i perímetre

Suricata s'ha recolzat sobre pfSense perquè pfSense ja és la peça central de xarxa d'Hefesto. Abans de parlar d'IDS/IPS, cal tenir clar que el firewall és qui defineix la segmentació, el NAT, les regles entre zones i l'accés des de l'exterior.

A la WAN, pfSense rep el trànsit que arriba des del router físic. A la LAN hi ha serveis interns i administració. A la DMZ hi ha serveis més exposats, com el servidor web. També hi ha accés remot per OpenVPN. Aquesta separació fa que sigui possible aplicar un model de defensa per capes.

El paper de Suricata no és obrir o tancar ports com fa el firewall. El seu paper és inspeccionar el trànsit que passa per les interfícies i detectar patrons sospitosos. Per exemple, un firewall pot permetre HTTP cap a un servidor web, però Suricata pot detectar que dins d'aquell HTTP hi ha un patró d'atac o un escaneig.

- pfSense controla regles, NAT i separació de xarxes.
- Suricata inspecciona el trànsit permès o visible.
- La DMZ és la zona amb més risc perquè conté serveis exposats.
- La LAN necessita vigilància però sense bloquejos agressius.
- La VPN s'ha de controlar perquè és una entrada d'administració remota.

Aquesta combinació firewall + IDS/IPS és més realista que confiar només en una de les dues parts. El firewall redueix superfície d'atac i Suricata aporta detecció i evidències.

Per això tota la configuració posterior depèn d'una base de firewall ben pensada.

6. Instal·lació i configuració inicial de Suricata

La instal·lació de Suricata s'ha fet des del gestor de paquets de pfSense. Aquest enfocament és còmode perquè permet administrar el servei des de la interfície web del firewall, sense haver de mantenir una màquina Linux independent només per IDS/IPS.

Després de la instal·lació, la configuració principal passa per activar Suricata en les interfícies que ens interessin, definir el mode de funcionament, habilitar logs, seleccionar categories de regles i revisar alertes. No s'ha tractat com un simple "activar i ja està", perquè una mala configuració pot generar massa soroll o bloquejar trànsit necessari.

En el projecte s'ha treballat especialment la vigilància de la DMZ, ja que és la zona on hi ha serveis que poden rebre connexions des de fora. També s'ha valorat la WAN i la LAN, però amb criteris diferents. A la WAN pot aparèixer molt soroll d'Internet; a la LAN no volem bloquejar trànsit intern legítim sense motiu.

Configuració	Objectiu
Interfícies	Decidir on es veu i inspecciona el trànsit

Categories de regles	Detectar atacs, escanejos i malware
Alertes	Registrar coincidències i severitat
IPS / bloqueig	Aplicar prevenció quan el risc és clar
Logs	Permetre anàlisi posterior i enviament a Zabbix

La configuració inicial ha estat el punt de partida. El valor real del projecte ve després, quan les alertes es revisen, es proven i s'ajusten segons el comportament de la xarxa.

7. Polítiques de seguretat per WAN, LAN i DMZ

Una de les decisions més importants ha estat no aplicar la mateixa política a totes les zones. Cada interfície té un risc i un comportament diferent. La DMZ necessita més vigilància perquè conté serveis exposats. La LAN necessita control, però evitant tallar operacions internes. La WAN pot generar molt soroll i cal filtrar amb criteri.

En la DMZ s'ha donat més pes a detecció d'escanejos, intents contra serveis web i comportaments propis d'un atac extern. Aquesta zona és la més interessant per validar Suricata perquè és on els serveis poden estar accessibles des de fora mitjançant NAT o reverse proxy.

A la WAN es va provar activar inspecció i prevenció, però cal anar amb compte. Quan es van activar determinades opcions d'IPS també a la WAN, es va observar que podia començar a bloquejar escanejos o trànsit cap al servidor DMZ. Això demostra que Suricata funciona, però també que s'ha de revisar l'impacte de cada canvi.

A la LAN, l'objectiu és detectar comportaments interns estranys: un host que escaneja, intents repetits, comunicacions anòmales o activitat que podria indicar compromís. No interessa convertir la LAN en una zona plena de falsos positius.

Zona	Criteri aplicat
DMZ	Vigilància forta sobre serveis exposats
WAN	Control d'entrada i reducció de soroll
LAN	Detecció interna sense bloquejar per defecte
VPN	Control d'accessos remots i intents repetits

La política final busca equilibri: protegir sense trencar el funcionament normal del laboratori.

8. Regles, signatures i categories utilitzades

Suricata funciona principalment amb regles i signatures. Aquestes regles descriuen patrons que poden indicar escaneig, malware, explotació, activitat de command and control, intents d'intrusió o tràfic que no encaixa amb una comunicació normal.

En un projecte com Hefesto no interessa activar-ho tot sense criteri. Si s'activen massa categories, poden aparèixer moltes alertes sense valor i és més difícil veure el que realment importa. Per això s'han seleccionat categories relacionades amb escaneig, malware, attack response, comportaments sospitosos i trànsit cap a serveis exposats.

També s'ha revisat la severitat de les alertes. No és el mateix una alerta informativa que una coincidència relacionada amb un intent clar d'atac. Aquesta diferència és important per prioritzar la resposta i no tractar tot com si fos crític.

- Regles relacionades amb scan i reconeixement.
- Categories d'attack response per detectar possibles respostes d'equips compromesos.
- Regles de malware i tràfic sospitos.
- Detecció sobre HTTP i serveis exposats a la DMZ.
- Revisió de falsos positius abans de passar a bloqueig permanent.

La feina amb signatures no és només activar llistes. Cal mirar què detecten, contra quin host, amb quina freqüència i si la detecció té sentit dins del nostre disseny de xarxa.

Aquesta és la diferència entre tenir Suricata instal·lat i tenir Suricata ajustat a l'entorn.

9. Logs, alertes i enviament cap a Zabbix

Els logs són una part central del projecte perquè permeten justificar què ha detectat Suricata. Una alerta sense registre no serveix gaire en una memòria ni en una resposta a incidents. Per això s'ha treballat la generació de logs a pfSense i la seva recepció posterior a Zabbix.

Suricata pot generar alertes amb informació com IP origen, IP destinació, ports, SID de la regla, categoria, severitat i hora. Aquesta informació permet entendre si l'esdeveniment és un escaneig, un intent contra un servei o només una coincidència poc important.

S'ha activat l'enviament de logs des de pfSense cap al servidor Zabbix mitjançant rsyslog. A Zabbix es reben en `/var/log/pfsense.log` i després es creen ítems per llegir logs generals o filtrar línies relacionades amb Suricata. Això permet veure alertes de seguretat al mateix dashboard on es revisa la salut de sistemes.

Camp d'alerta	Valor per l'anàlisi
IP origen	Identificar atacant o host sospitos
IP destinació	Saber quin servei rep el trànsit
Port	Relacionar l'alerta amb el servei afectat
SID / regla	Entendre quin patró ha coincidit
Severitat	Prioritzar la resposta
Hora	Relacionar amb altres logs i incidents

Aquest punt connecta directament els miniprojectes de seguretat i monitorització. Suricata detecta i Zabbix ajuda a veure-ho dins de la visió general d'Hefesto.

10. Simulació d'atacs i comprovació de funcionament

Per validar Suricata no n'hi ha prou amb veure el servei actiu. S'han de provocar situacions controlades i comprovar si el sistema les detecta. La prova més representativa ha estat simular escanejos contra serveis de la DMZ, que és un comportament típic de reconeixement abans d'un atac.

Aquest tipus de prova permet validar diferents coses alhora: que el trànsit passa per la interfície inspeccionada, que les regles actives detecten el patró, que l'alerta apareix a pfSense i que el log pot arribar a Zabbix. Si una d'aquestes parts falla, la cadena de detecció queda incompleta.

També s'ha comprovat l'efecte de tenir IPS actiu en determinades interfícies. Quan Suricata bloqueja un escaneig, això confirma capacitat de prevenció, però també obliga a revisar si el bloqueig és correcte o si pot afectar proves legítimes del laboratori.

- Escaneig de ports contra serveis de la DMZ.
- Revisió d'alertes generades per Suricata.
- Comprovació de logs a pfSense i recepció a Zabbix.
- Anàlisi de si la regla només ha d'alertar o també bloquejar.
- Comparació del comportament abans i després dels ajustos.

Aquestes proves donen valor a la documentació perquè demostrin que el sistema no és només una configuració, sinó una eina que respon davant comportaments reals.

11. Problemes trobats i ajustos de Suricata

Durant el projecte han aparegut problemes que són normals quan es treballa amb IDS/IPS. El primer és el soroll: Suricata pot generar moltes alertes, especialment si es miren interfícies exposades o categories massa àmplies. No totes les alertes són igual d'importantes, i cal revisar-les amb calma.

També s'ha vist que activar prevenció en zones sensibles pot tallar connexions o proves legítimes. En algun moment, activar IPS també sobre la WAN va fer que Suricata bloquegés trànsit relacionat amb escanejos cap al servidor DMZ. Això és bo des del punt de vista de detecció, però si no es controla pot donar la sensació que "la xarxa falla" quan en realitat l'IPS està actuant.

Per aquest motiu es va descartar complicar massa el projecte amb configuracions inline més agressives i es va mantenir un enfocament més estable. Primer veure alertes, després ajustar regles i finalment bloquejar només allò que sigui clar. Aquesta decisió evita perdre connectivitat contínuament mentre es documenta i valida el projecte.

Un altre problema és la interpretació de signatures. Una alerta com `Applayer Detect protocol only one direction` pot indicar que Suricata només ha vist una direcció de la conversa, però no necessàriament significa un atac directe. Aquest tipus d'avisos s'han de llegir amb context.

Problema	Resposta aplicada
Massa alertes	Filtrar categories i revisar severitats
Possibles falsos positius	Validar abans de bloquejar
Bloquejos inesperats	Ajustar IPS per interfície
Alertes difícils d'interpretar	Analitzar IP, port, regla i context

Aquests ajustos són part important del resultat final perquè mostren criteri tècnic i no només instal·lació.

12. Resposta a incidents i actuació pràctica

La resposta a incidents s'ha plantejat de forma senzilla però útil. Quan apareix una alerta, el primer pas no és bloquejar sense pensar, sinó entendre què ha passat: quin és l'origen, quin és el destí, quin port està implicat, quina regla ha saltat i si el servei afectat és realment crític. Si l'alerta és un escaneig contra la DMZ, la resposta pot ser bloquejar temporalment la IP, revisar regles del firewall, comprovar logs del servidor web i verificar si hi ha més intents relacionats. Si és una alerta interna a la LAN, cal mirar si pot venir d'una màquina compromesa o d'una prova legítima del laboratori.

També s'ha valorat la resposta des del punt de vista de continuïtat. En un entorn amb Proxmox, NAS i serveis virtualitzats, una resposta a incident no és només bloquejar una IP. També pot implicar tenir còpies, revisar snapshots, migrar serveis o recuperar una VM si alguna cosa queda afectada.

Fase	Acció
Detecció	Revisar alerta, severitat i regla
Anàlisi	Comprovar origen, destí, port i servei
Contenció	Bloquejar o limitar trànsit si cal
Correcció	Ajustar firewall, servei o regla
Seguiment	Mirar si es repeteix i documentar evidències

La idea és que Suricata sigui una eina per decidir millor, no només una llista d'alertes. Una bona resposta depèn de combinar Suricata, pfSense, Zabbix i el coneixement de la topologia d'Hefesto.

13. Integració amb sistemes complementaris

Suricata no treballa sol dins d'Hefesto. Forma part d'un conjunt de sistemes que es complementen. pfSense aplica regles i NAT, OpenVPN dona accés remot, Proxmox allotja

serveis i Zabbix centralitza l'estat general i els logs. Aquesta integració és la que dona sentit al miniprojecte.

La combinació amb pfSense és la més directa. pfSense permet controlar quin trànsit arriba a cada zona i Suricata analitza el trànsit que passa. Així es manté un model de defensa per capes: primer reduir la superfície d'exposició i després inspeccionar allò que sí circula.

La integració amb Zabbix també és important. Els logs de Suricata no es queden només a pfSense, sinó que poden veure's al mateix entorn on es revisa si el servidor web està actiu, si Proxmox té problemes o si el NAS està saturat. Això facilita relacionar seguretat i disponibilitat.

- pfSense com a firewall i punt d'inspecció.
- OpenVPN com a accés remot controlat.
- Proxmox com a plataforma on viuen els serveis vigilats.
- Servidor web DMZ com a objectiu de proves i monitorització.
- Zabbix com a eina de visualització de logs i alertes.

També es podria ampliar en el futur amb un SIEM dedicat, però per l'abast del projecte, la unió Suricata + pfSense + Zabbix és suficient i queda ben justificada.

Això converteix Suricata en una peça integrada dins de l'ecosistema Hefesto, no en un servei separat sense context.

14. Validació final del sistema IDS/IPS

La validació final s'ha basat en comprovar que Suricata detecta trànsit sospitos, genera alertes, deixa logs i permet prendre decisions. La part més important és que les proves s'han fet dins de la pròpia topologia del projecte, no en un exemple extern sense relació amb Hefesto.

S'ha validat especialment la detecció d'escanejos cap a serveis de la DMZ. Aquesta prova és coherent perquè un atacant normalment començaria intentant descobrir ports i serveis oberts abans d'intentar explotar-los. Si Suricata detecta aquesta fase, ja aporta valor com a sistema d'alerta primerenca.

També s'ha comprovat que les alertes es poden consultar des de pfSense i que els logs poden arribar a Zabbix. Aquesta traçabilitat és important perquè permet relacionar una alerta amb altres dades del sistema: estat del servidor web, problemes de xarxa, caiguda d'un servei o activitat anòmla en el mateix moment.

Aspecte validat	Resultat
Servei Suricata	Instal·lat i actiu sobre pfSense
Interfícies	Inspecció aplicada segons zones
Regles	Categories ajustades al risc de l'entorn

Alertes	Generació de registres consultables
Zabbix	Recepció i visualització de logs
Resposta	Criteri definit per analitzar i actuar

El sistema queda validat perquè compleix les tres parts principals: detectar, registrar i ajudar a respondre. No és una configuració perfecta ni definitiva, però sí una base sòlida per un entorn ASIX realista.

15. Resultat final, límits i millores futures

El resultat final de la part Suricata és un sistema IDS/IPS integrat dins de pfSense, adaptat a la topologia d'Hefesto i connectat amb la monitorització de Zabbix. Aquesta integració dona visibilitat sobre intents d'escaneig, tràfic sospitós i possibles comportaments anòmals dins de la xarxa.

El projecte ha demostrat que un firewall per si sol no dona tota la informació necessària. pfSense pot permetre o bloquejar trànsit segons regles, però Suricata permet veure patrons dins del trànsit. Aquesta diferència és clau quan es vol passar d'un filtratge bàsic a una seguretat més completa.

També s'ha après que un IDS/IPS necessita ajustos. No es pot activar tot i assumir que tot funcionarà bé. Cal revisar falsos positius, entendre alertes, decidir on aplicar IPS i evitar bloquejar trànsit legítim. Aquesta part és tan important com la instal·lació.

Punt	Estat final
IDS/IPS	Suricata desplegat sobre pfSense
Polítiques	Separades per WAN, LAN, DMZ i VPN
Logs	Disponibles a pfSense i exportables a Zabbix
Proves	Escanejos i alertes validades
Resposta	Procediment bàsic d'anàlisi i contenció

Com a millores futures es podria afegir un SIEM, dashboards específics per severitat, correlació automàtica d'alertes, bloquejos temporals més controlats i revisió periòdica de regles. També es podria reforçar la resposta amb backups o snapshots automàtics davant incidents greus.

Tot i aquests límits, la solució compleix l'objectiu: reforçar la seguretat d'Hefesto amb detecció, prevenció controlada i evidències útils per analitzar incidents.

Monitorització amb Zabbix

1. Objectiu real de la monitorització a Hefesto

Aquest apartat resumeix què s'ha aconseguit amb Zabbix sense convertir-ho en un manual pas a pas.

La part de Zabbix del projecte Hefesto no s'ha plantejat només com instal·lar una eina i deixar-la oberta al navegador. El que s'ha fet és muntar un punt central de control per entendre què passa dins de la infraestructura: servidors, Proxmox, NAS, serveis web, pfSense, Suricata i alertes relacionades amb la xarxa.

Abans d'aquesta part, qualsevol problema s'havia de revisar entrant a cada equip per separat. Si fallava un node, un servei web o el NAS, calia mirar-ho manualment i anar relacionant dades. Amb Zabbix hem canviat aquest enfocament: ara el sistema recull mètriques, genera avisos i mostra l'estat general del laboratori des d'un únic lloc.

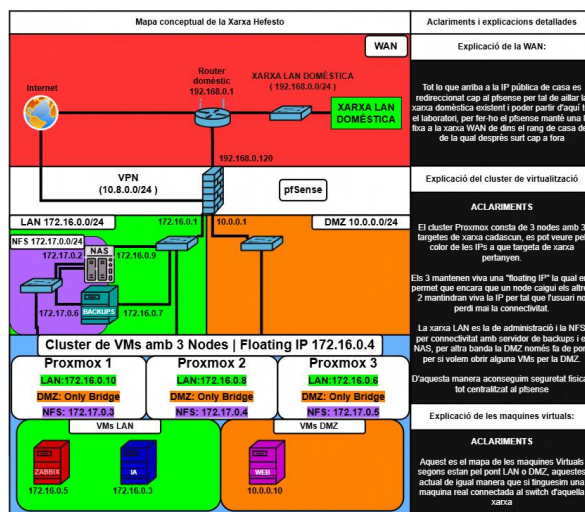
El servidor Zabbix s'ha situat a la LAN amb IP fixa 172.16.0.5. Aquesta decisió és important perquè la monitorització ha de ser estable i no pot dependre d'una IP que canviï. També permet que la resta de màquines i serveis coneguin sempre quin és el servidor que rep mètriques o logs.

El projecte ha barrejat diferents formes de monitorització: agent Zabbix per a Linux, SNMP per al NAS, API per al clúster Proxmox, scripts propis per al servidor web i rsyslog per recollir informació de pfSense i Suricata. Aquesta barreja és més realista que dependre d'un únic mètode, perquè cada servei del laboratori necessita una forma diferent de ser observat.

- Monitorització de disponibilitat i recursos dels equips principals.
- Integració de TrueNAS per SNMP i Proxmox per API.
- Scripts propis per saber l'estat real del servidor web de la DMZ.
- Recepció de logs de pfSense i Suricata per tenir seguretat i xarxa en el mateix panell.
- Dashboards i reports per convertir les dades en informació útil.

El resultat final és una memòria de treball on Zabbix queda com una eina realment integrada dins d'Hefesto i no com una instal·lació aïllada.

2. Arquitectura de xarxa i posició del servidor Zabbix



Imatge del repositori: diagrama general del laboratori Hefesto.

La infraestructura d'Hefesto està separada en diferents zones: WAN, LAN, DMZ, xarxa d'emmagatzematge i entorn de virtualització sobre Proxmox. En aquest context, Zabbix s'ha col·locat a la LAN perquè és una zona interna i controlada, però amb capacitat per arribar als serveis que volem supervisar.

Aquesta ubicació té sentit perquè el servidor de monitorització no ha d'estar exposat directament a Internet. Zabbix necessita visibilitat sobre la infraestructura, però no ha de convertir-se en un servei públic. Per això queda darrere de pfSense i treballa com a eina interna d'administració.

Element	Ubicació / funció	Com es monitoritza
Zabbix	LAN - servidor central	Agent local i interfície web
TrueNAS	Xarxa interna / emmagatzematge	SNMP v3
Proxmox	Clúster de virtualització	API token i IP virtual
Servidor web	DMZ	Agent Zabbix + script propi
pfSense / Suricata	Perímetre i seguretat	rsyslog i ítems de logs

Una de les claus del disseny ha estat no perdre el sentit de la topologia. No ens interessa només veure gràfiques: ens interessa saber si falla el NAS que guarda màquines, si cau un node Proxmox, si Apache no respon a la DMZ o si Suricata està generant alertes de seguretat. Per tant, Zabbix funciona com una capa de visibilitat sobre tota la infraestructura Hefesto, respectant la separació de xarxes i aprofitant els protocols que ja encaixen amb cada servei.

3. Instal·lació base i preparació del servidor

La instal·lació de Zabbix s'ha fet sobre Debian 12, desplegat com a màquina virtual dins de Proxmox. El primer criteri ha estat deixar un sistema base net, estable i amb una configuració de xarxa fixa. En monitorització això és bàsic, perquè si el servidor canvia d'IP o queda mal resolt per DNS, la resta d'integracions deixen de tenir sentit.

La interfície principal de la VM s'ha configurat amb la IP 172.16.0.5, gateway 172.16.0.1 i DNS internes de la xarxa. Amb això Zabbix queda dins de la LAN i pot comunicar-se amb pfSense, Proxmox, TrueNAS i la DMZ segons les regles de xarxa configurades.

A nivell de paquets s'ha instal·lat el servidor Zabbix, la interfície web, Apache, PHP, l'agent i MariaDB. La base de dades s'ha preparat manualment creant la base `zabbix`, l'usuari corresponent i important l'esquema inicial. Això permet que Zabbix guardi hosts, mètriques, triggers, problemes, gràfiques, usuaris i configuració general.

També s'ha editat `zabbix\_server.conf` per definir correctament la connexió amb MariaDB. Aquest punt és delicat perquè un error a `DBName`, `DBUser` o `DBPassword` fa que el servei no pugui arrencar. Per això es va revisar abans d'iniciar el servei i continuar amb l'assistent web.

Peça instal·lada	Funció dins del projecte
Debian 12	Sistema base del servidor de monitorització
MariaDB	Base de dades de Zabbix
Apache + PHP	Frontend web de Zabbix
Zabbix server	Procés principal que recull i avalua dades
Zabbix agent	Monitorització local del propi servidor

La instal·lació no ha estat documentada com una llista de comandes sense context, sinó com la base sobre la qual després s'han connectat els serveis reals de l'entorn.

4. Configuració inicial de la interfície i criteris de treball

Després de la instal·lació del backend, s'ha completat l'assistent web de Zabbix. Aquí es va configurar la connexió amb la base de dades, el nom del servidor, la zona horària i la interfície de treball. La zona horària `Europe/Madrid` és important perquè les alertes han de quadrar amb els logs de pfSense, Suricata, Proxmox i els sistemes Linux.

Un problema habitual en projectes d'aquest tipus és tenir hores diferents entre sistemes. Si Zabbix mostra una alerta amb una hora i Proxmox o pfSense una altra, després costa molt reconstruir què ha passat. Per això s'ha tingut en compte la coherència horària des del principi. També s'ha deixat la interfície en mode fosc, no només per gust visual, sinó perquè en una eina de monitorització es treballa moltes vegades revisant pantalles amb molts colors,

gràfiques i severitats. El tema fosc ajuda a tenir una lectura més còmoda i encaixa amb la forma en què s'ha anat documentant el projecte.

A nivell d'organització, els hosts s'han creat amb noms visibles clars. No és el mateix tenir una IP solta que veure `NAS-Hefesto-1`, `ClusterHefesto` o `WEB-Hefesto`. Aquesta decisió sembla petita, però quan hi ha alertes o dashboards fa que la interpretació sigui molt més ràpida.

- Zona horària coherent amb la resta de sistemes.
- Noms visibles entenedors per evitar confusions.
- Tema fosc per facilitar la lectura de problemes i gràfiques.
- Separació mental entre monitorització de serveis, xarxa, storage i seguretat.
- Base preparada per construir dashboards i informes sense desordre.

Aquestes decisions inicials han servit per no acabar amb un Zabbix ple d'ítems però difícil d'entendre.

5. Alta de hosts, grups i autodescobriment

Un cop el servidor funcionava, el següent pas ha estat ordenar la infraestructura dins de Zabbix. S'han creat grups de hosts per separar els diferents tipus d'equip: Linux, NAS, Proxmox, serveis web i elements relacionats amb logs. Aquesta separació ajuda a filtrar problemes i a crear dashboards més útils.

També s'ha activat una regla de discovery sobre la LAN. La idea és que Zabbix pugui detectar equips dins del rang 172.16.0.0/24 i ajudar a localitzar nous dispositius o serveis que apareguin durant el projecte. En un laboratori amb moltes màquines virtuals, això evita dependre completament d'altres manuals.

El discovery no s'ha deixat com una solució màgica. Serveix per detectar, però després cal aplicar plantilles, revisar noms, ordenar grups i decidir què s'ha de monitoritzar de veritat. Si es deixa tot automàtic sense criteri, Zabbix pot omplir-se de dades que no aporten gaire.

Decisió	Motiu
Crear grups de hosts	Separar alertes per tipus de dispositiu
Activar discovery LAN	Detectar equips nous o serveis disponibles
Usar plantilles	Evitar crear totes les mètriques manualment
Revisar ítems actius	No carregar el sistema amb informació inútil
Noms visibles	Identificar ràpidament l'origen dels problemes

Aquesta part ha estat clau perquè la monitorització sigui escalable. La feina no és només afegir hosts, sinó tenir una estructura que permeti entendre la xarxa d'una ullada.

6. Monitorització del NAS TrueNAS per SNMP

El NAS TrueNAS és una peça important del projecte perquè forma part de l'emmagatzematge utilitzat per les màquines virtuals i serveis. Per aquest motiu s'ha integrat a Zabbix amb SNMP, que és el mètode més adequat per obtenir informació d'equips de xarxa i storage sense instal·lar agents a tot arreu.

S'ha configurat el host `NAS-Hefesto-1` amb la IP 172.16.0.9 i s'ha vinculat amb una plantilla SNMP compatible amb TrueNAS/FreeNAS. Al costat de TrueNAS s'ha activat el servei SNMP i s'ha optat per SNMP v3, amb autenticació i privacitat, perquè és més coherent amb un projecte de seguretat i no deixa la informació exposada com passaria amb una comunitat simple.

Durant aquesta integració van aparèixer alertes que no sempre representaven un problema real. Per exemple, algunes temperatures de discs o estats de pools podien sortir estranys pel fet de treballar en un entorn virtualitzat. Això ens va obligar a revisar triggers i no acceptar cegament tot el que diu una plantilla.

Punt monitoritzat	Valor que aporta
Disponibilitat del NAS	Saber si l'equip respon i està accessible
CPU i memòria	Detectar si el NAS va just de recursos
Pool i espai	Controlar l'emmagatzematge de les VMs
Alertes SNMP	Rebre avisos de l'estat intern del NAS
Triggers ajustats	Reduir falsos positius per virtualització

La conclusió d'aquesta part és clara: les plantilles ajuden molt, però s'han d'adaptar al context. En un entorn virtual, no tot el que surt com a alerta crítica ho és realment.

7. Integració del clúster Proxmox i problema d'alertes duplicades

La integració de Proxmox ha estat una de les parts més importants i també una de les més interessants del projecte. No es tractava només de monitoritzar un servidor, sinó un clúster amb diversos nodes i màquines virtuals que poden moure's entre ells.

Inicialment es va plantejar afegir els tres nodes per separat, però això generava un problema clar: alertes duplicades o triplicades. Cada node podia informar de l'estat dels altres i això feia que el panell quedés brut, amb avisos repetits que no ajudaven a entendre millor la situació.

L'altra opció era afegir només un node, però això trencava la idea d'alta disponibilitat. Si el node principal que consulta Zabbix cau o queda inaccessible, Zabbix perd el punt de consulta encara que la resta del clúster continuï funcionant. Per solucionar-ho es va treballar amb Keepalived i una IP virtual 172.16.0.4 compartida entre nodes.

Aquesta IP virtual permet que Zabbix consulti el clúster per un únic punt lògic. Si el node principal falla, un altre pot assumir la IP. Això encaixa millor amb la idea de monitoritzar un clúster com a conjunt i no com tres servidors inconnexos.

Problema	Solució aplicada
Alertes triplicades	Monitoritzar el clúster per una IP virtual
Caiguda del node principal	Keepalived amb MASTER/BACKUP
Accés API segur	Usuari i token específic per Zabbix
Mètriques de nodes	Plantilla Proxmox amb macros PVE

Aquesta decisió és de les que més valor aporta a la memòria, perquè mostra que no només s'ha seguit una plantilla, sinó que s'ha resolt un problema real de disseny.

8. Recepció de logs de pfSense i Suricata amb rsyslog

A més de monitoritzar sistemes, s'ha volgut unir la part de seguretat amb la part de monitorització. Per això s'ha configurat rsyslog al servidor Zabbix per rebre logs des de pfSense, incloent alertes o registres relacionats amb Suricata.

El plantejament ha estat crear un fitxer de configuració a `/etc/rsyslog.d/` indicant que els logs provinents de pfSense, amb IP 172.16.0.1, es guardin en `/var/log/pfsense.log`. Després s'han creat ítems a Zabbix per llegir aquest fitxer i filtrar informació concreta.

Aquesta integració és útil perquè permet veure en un mateix entorn tant dades de salut de sistemes com informació de seguretat. Per exemple, si Suricata detecta un escaneig o pfSense genera un avís, es pot veure al dashboard juntament amb l'estat dels hosts, el NAS o el clúster Proxmox.

El problema aquí és que els logs poden generar molt soroll. No tot el que apareix en un log és una incidència greu. Per això es va separar la informació de pfSense i Suricata en ítems diferents i es va plantejar el dashboard com una vista de consulta, no com una alerta crítica per cada línia.

- Activació de recepció de logs remots per TCP/UDP al port 514.
- Separació dels logs de pfSense en un fitxer propi.
- Creació d'ítems Zabbix per llegir logs generals i filtrar Suricata.
- Validació amb `tail` per confirmar que les línies arribaven al servidor.
- Ús posterior als dashboards en format text.

Aquesta part deixa Zabbix com un punt de correlació bàsic entre infraestructura i seguretat.

9. Monitorització personalitzada del servidor web de la DMZ

El servidor web de la DMZ s'ha monitoritzat amb un enfocament més personalitzat. En lloc de limitar-nos a saber si la màquina respon a ping, s'ha volgut saber si Apache està actiu, si la web retorna un HTTP correcte, si la configuració és vàlida i si hi ha errors recents.

Per fer-ho s'ha instal·lat l'agent de Zabbix al servidor web i s'ha creat un script propi anomenat `hefesto_web_health.sh`. Aquest script executa comprovacions locals i retorna valors que després Zabbix recull amb `UserParameter`. Això dona molta més informació que una plantilla genèrica.

Aquesta decisió és important perquè la DMZ és una zona més exposada. Pot passar que la màquina estigui encesa, però Apache estigui caigut o la web retorni errors 500. Sense una comprovació de servei, Zabbix podria marcar el host com disponible encara que la web realment no estigués funcionant.

Key creada	Què comprova
hefesto.web.apache	Si Apache està actiu
hefesto.web.http_code	Codi HTTP retornat localment
hefesto.web.configtest	Validació de configuració Apache
hefesto.web.errors	Errors recents al log
hefesto.web.5xx	Respostes HTTP de servidor
hefesto.web.disk_www	Ús del directori /var/www
hefesto.web.score	Salut general del servei web
hefesto.web.summary	Resum textual de l'estat

Amb aquesta part, el servidor web queda monitoritzat des del punt de vista real del servei, no només des del punt de vista de la màquina.

10. Items, health score i resum del servei web

El script del servidor web no només retorna dades aïllades. També calcula un valor de salut general, el `health score`, que serveix per resumir en un número si la web està bé o si hi ha alguna cosa que comença a fallar.

La lògica és senzilla però efectiva: es parteix d'un valor alt i es van restant punts si Apache no està actiu, si el codi HTTP no és correcte, si la configuració falla, si hi ha massa errors o si el directori web està massa ple. Això permet detectar situacions degradades encara que no hi hagi una caiguda total.

També s'ha creat `hefesto.web.summary`, que retorna una línia de text amb diferents dades juntes. Aquest ítem no és tan bo per fer gràfiques, però és molt útil en un dashboard perquè permet llegir ràpidament l'estat complet del servei sense entrar a cada ítem.

Valor	Interpretació
Apache = 1	Servei principal actiu
HTTP = 200	La web respon correctament
Config = 1	Apache té configuració vàlida
5xx baix	No hi ha errors interns repetits
Disc < 80%	El directori web no està al límit
Score > 70	Salut general acceptable

Aquesta forma de treball és molt útil en projectes reals perquè no tot és blanc o negre. Un servei pot continuar viu però anar malament. Amb el score es pot veure aquesta degradació abans que el problema sigui crític.

També cal destacar que aquest punt demostra personalització. Zabbix no només s'ha fet servir amb plantilles, sinó que s'ha adaptat al servei concret que tenim dins d'Hefesto.

11. Triggers, llindars i reducció de soroll

Un dels punts més importants en monitorització és configurar alertes que siguin útils. Si tot genera alerta, al final no es mira res. Per això s'han revisat triggers de les plantilles i també s'han creat triggers propis per al servidor web.

En Proxmox es va detectar que algunes màquines virtuals apagades o reiniciades generaven avisos que podien ser molestos. En un laboratori, no sempre és un problema que una VM estigui apagada. Per aquest motiu es van ajustar severitats i criteris per deixar com a important allò que realment afecta el servei: nodes caiguts, recursos molt elevats o serveis crítics fora de línia.

En TrueNAS es van desactivar o ajustar triggers de temperatura perquè l'entorn virtual donava informació poc fiable. En canvi, es van mantenir avisos relacionats amb CPU, memòria o espai, que sí que aporten valor.

Per al servidor web s'han creat triggers directes: Apache no actiu, HTTP incorrecte, configuració Apache invàlida, health score baix, massa errors 5xx i ús alt de `/var/www``. Aquests triggers estan pensats per detectar problemes que afecten l'usuari final o la disponibilitat del servei.

Trigger	Criteri aplicat
Apache no actiu	Últim valor de servei = 0
Web no respon bé	HTTP inferior a 200 o superior/igual a 400
Config Apache incorrecta	Configtest = 0
Health score baix	Score inferior a 70
Errors 5xx elevats	Més de 5 errors recents

Disc web alt

Ús de /var/www superior al 80%

La idea final ha estat prioritzar qualitat d'alerta per sobre de quantitat.

12. Dashboards operatius i vista general del sistema

Els dashboards han estat la part on totes les dades recollides comencen a tenir sentit visual. No es tracta només de posar gràfiques perquè sí, sinó de construir una vista que ajudi a entendre l'estat de la infraestructura Hefesto en poc temps.

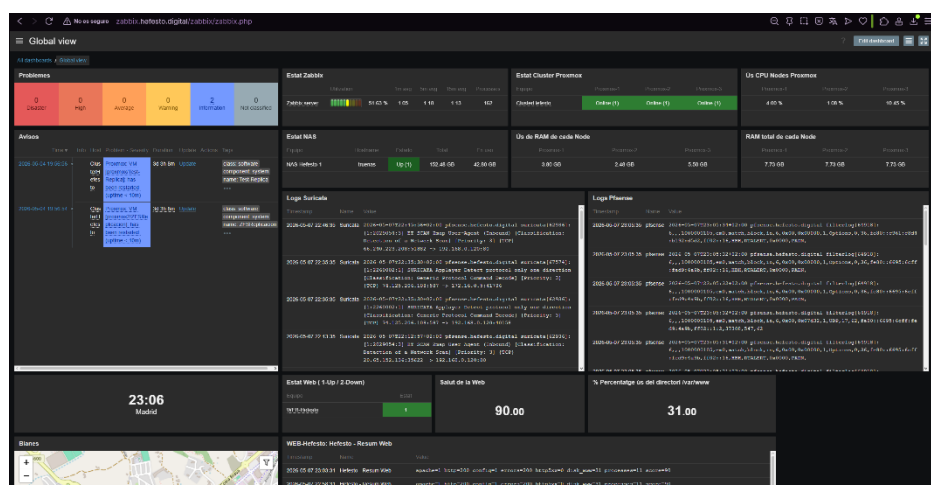
El dashboard principal reuneix problemes actius, estat del servidor Zabbix, estat dels nodes Proxmox, consum de CPU i RAM, informació del NAS, logs de pfSense i Suricata, i també l'estat del servidor web de la DMZ. Això permet veure si una incidència és només d'un servei o si està relacionada amb una part més gran de la infraestructura.

Per exemple, si el servidor web no respon, podem revisar al mateix panell si Apache està actiu, si el host està disponible, si Proxmox mostra problemes, si el NAS té alertes o si Suricata ha registrat activitat sospitosa. Aquesta relació entre dades és el que fa útil el dashboard.

- Widgets de problemes actius i avisos recents.
- Estat dels nodes Proxmox amb informació separada per node.
- CPU, RAM i memòria total per contextualitzar consum.
- Estat del NAS i ús de la pool important.
- Plain text per logs de Suricata, pfSense i resum web.
- Indicadors de salut per al servidor web de la DMZ.

També s'han afegit widgets com geolocalització i hora del servidor, no com a part crítica, sinó per completar la vista i fer-la més còmoda durant la revisió.

El resultat és una pantalla de control que pot servir tant per documentar el projecte com per operar realment el laboratori.



13. Informes periòdics, correu i problemes trobats

A més dels dashboards en temps real, s'ha explorat la part d'informes periòdics de Zabbix. La idea és generar un PDF amb l'estat del dashboard i enviar-lo de manera programada per correu. Això és útil perquè no sempre cal entrar manualment a la plataforma per revisar l'estat del sistema.

Per activar aquesta part s'ha instal·lat `zabbix-web-service` i Google Chrome, ja que Zabbix utilitza aquest component per renderitzar dashboards i convertir-los en informes. També s'han ajustat paràmetres com `WebServiceURL` i `StartReportWriters` dins de la configuració del servidor.

Un dels problemes trobats va ser que Chrome no podia iniciar correctament perquè el servei no tenia permisos o directori adequat per crear fitxers sota `/var/lib/zabbix`. Aquest error és important perquè demostra que els reports no depenen només de marcar una opció al frontend: també cal preparar bé el sistema on corre el servei.

També s'ha configurat un compte de correu del domini `hefesto.digital`, aprofitant el servei de Nominalia, per enviar reports de manera més professional. Aquesta part encaixa amb la idea de projecte final, perquè no queda com un correu personal improvisat, sinó com una notificació pròpia de la infraestructura.

Component	Finalitat
zabbix-web-service	Generar informes dels dashboards
Google Chrome	Renderitzar el contingut web del report
Media type SMTP	Enviar correus des de Zabbix
Usuari Admin / media	Definir receptors dels informes
Informe setmanal Hefesto	Resum periòdic de l'estat del sistema

Aquesta part deixa preparada una monitorització no només reactiva, sinó també orientada a seguiment periòdic.

14. Proves de validació realitzades

La validació no s'ha quedat en veure ítems en verd. S'han fet proves controlades per confirmar que Zabbix detecta problemes reals dins de l'entorn Hefesto. Aquest punt és important perquè una monitorització que no s'ha provat pot donar una falsa sensació de seguretat.

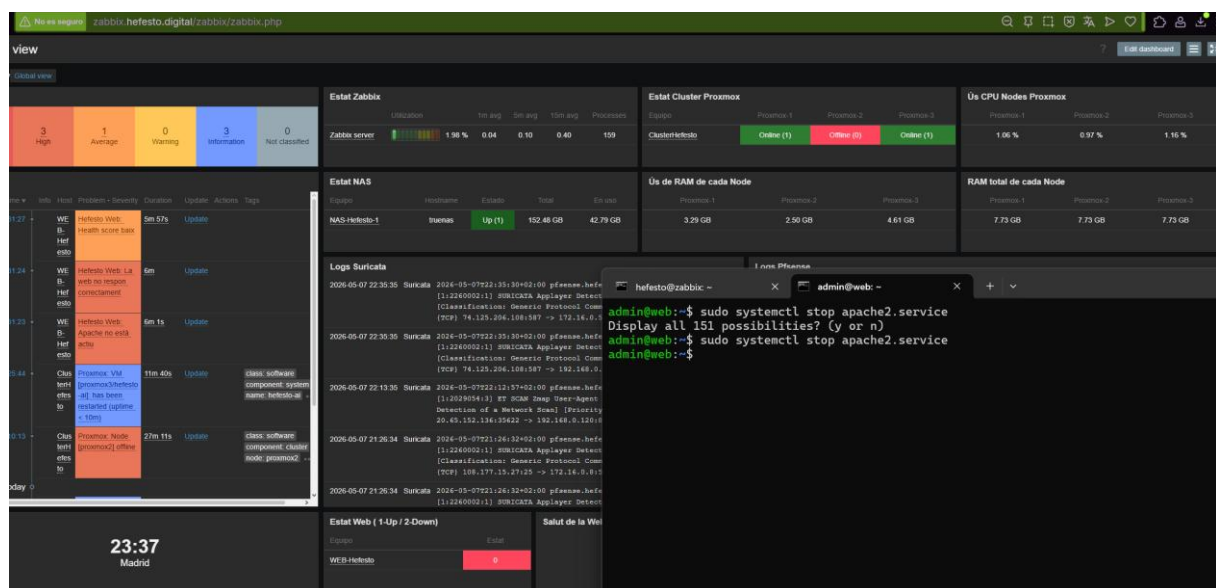
La primera prova important ha estat apagar un node Proxmox, concretament proxmox2. Amb això s'ha comprovat que el clúster mostra el node caigut i que Zabbix genera un problema de severitat alta. Aquesta prova valida la integració per API i també el plantejament amb la IP virtual del clúster.

Una altra prova ha estat forçar un ús elevat de CPU en una màquina virtual, com la VM web, per veure si els llindars configurats generaven avisos. Aquest tipus de prova serveix per comprovar que Zabbix no només detecta caigudes totals, sinó també situacions de rendiment que poden anticipar problemes.

També s'ha aturat manualment el servei Apache al servidor web amb `systemctl stop apache2.service`. En aquest cas, Zabbix ha detectat que Apache no estava actiu, que la web no responia correctament i que el health score baixava. Aquesta és una de les proves més clares perquè valida el script propi.

- Caiguda d'un node Proxmox i detecció de node offline.
- Ús elevat de CPU per validar llindars de rendiment.
- Aturada d'Apache per comprovar triggers del servidor web.
- Revisió del dashboard abans i després de les incidències.
- Comprovació que logs de pfSense i Suricata continuaven arribant.

Aquestes proves confirmen que la solució és funcional i no només decorativa.



15. Problemes, ajustos i resultat final

Durant el projecte han aparegut problemes que han ajudat a millorar la solució final. El més clar ha estat la duplicació d'alertes en Proxmox. Afegir tres nodes per separat semblava lògic, però a la pràctica generava massa repetició. La solució amb Keepalived i IP virtual ha estat més neta i més professional.

També hi ha hagut problemes relacionats amb la xarxa de la VM Zabbix quan es movia entre nodes Proxmox. En alguns casos mantenia la IP però perdia connectivitat cap al gateway o Internet. Això recorda que en un entorn nested, les bridges, el mode promiscu i la coherència de xarxa entre nodes són igual d'importants que la configuració dins de la VM.

En TrueNAS, algunes alertes de temperatura o pool no eren del tot fiables pel context virtualitzat. Aquest punt ha obligat a ajustar triggers i entendre que una plantilla no sempre encaixa al 100% amb el laboratori.

En la part d'informes, el problema amb Chrome i permisos de `/var/lib/zabbix` també ha estat rellevant. No és un error de Zabbix com a tal, sinó de l'entorn d'execució del servei que genera els reports. Documentar-ho aporta valor perquè és un problema real que pot passar en desplegaments normals.

- No acceptar totes les alertes per defecte de les plantilles.
- Diferenciar problemes reals de falsos positius.
- Monitoritzar Proxmox com a clúster i no com nodes repetits.
- Controlar bé xarxa i bridges en un entorn nested.
- Validar reports i correu amb proves reals, no només configuració teòrica.

Aquests ajustos són els que fan que la memòria sigui més real i no una simple successió de captures.

Com a resultat final, Zabbix queda com una plataforma de monitorització centralitzada amb dades de sistemes, xarxa, virtualització, storage, serveis web i seguretat. S'ha integrat TrueNAS per SNMP, Proxmox per API, el servidor web amb scripts propis, pfSense i Suricata per logs, i el propi servidor Zabbix amb agent local.

Com a millores futures es podria afegir correlació avançada d'alertes, avisos automàtics per correu per tots els problemes crítics, mapes de xarxa més complets i integració amb Telegram o un SIEM. Tot i això, l'estat actual ja compleix l'objectiu principal: tenir control real de l'entorn i poder reaccionar abans que una incidència passi desapercebuda.

Bibliografia

Proxmox:

de No Solo Hacking:

https://www.youtube.com/playlist?list=PLznRNLIWBPwH5Li7Co2i57rUVhve7m_ZQ

Proxmox: <https://pve.proxmox.com/pve-docs/>

Docker:

Anthropic - Claude: <https://claude.ai/>

Docker Compose v2 (referència YAML): <https://docs.docker.com/compose/compose-file/>

Docker Swarm Mode: <https://docs.docker.com/engine/swarm/>

Docker Secrets: <https://docs.docker.com/engine/swarm/secrets/>

Kubernetes v1.30 (documentació oficial): <https://kubernetes.io/docs/home/>

kubectl referència de comandes: <https://kubernetes.io/docs/reference/kubectl/>

Minikube (entorn local Kubernetes): <https://minikube.sigs.k8s.io/docs/>

k3s (Kubernetes lleuger per a VMs): <https://docs.k3s.io/>

Suricata:

Suricata User Guide: <https://docs.suricata.io/>

Suricata Documentation: <https://suricata.io/documentation/>

Suricata GitHub oficial: <https://github.com/OISF/suricata>

Suricata Rules: <https://docs.suricata.io/en/latest/rules/index.html>

Suricata Rule Management: <https://docs.suricata.io/en/latest/rule-management/index.html>

Suricata EVE JSON Output: <https://docs.suricata.io/en/latest/output/eve/eve-json-output.html>

Suricata Quickstart Guide: <https://docs.suricata.io/en/latest/quickstart.html>

pfSense IDS/IPS: <https://docs.netgate.com/pfsense/en/latest/packages/snort/index.html>

pfSense Packages: <https://docs.netgate.com/pfsense/en/latest/packages/index.html>

Emerging Threats Rules: <https://rules.emergingthreats.net/>

OpenAI - ChatGPT: <https://chatgpt.com/>

Anthropic - Claude: <https://claude.ai/>

Zabbix:

Zabbix Documentation: <https://www.zabbix.com/documentation/current/en/manual>

Zabbix Official Website: <https://www.zabbix.com/>

Zabbix Installation: <https://www.zabbix.com/documentation/current/en/manual/installation>

Zabbix Integrations and Templates: <https://www.zabbix.com/integrations>

Zabbix Proxmox Integration: <https://www.zabbix.com/integrations/proxmox>

Zabbix TrueNAS Integration: <https://www.zabbix.com/integrations/truenas>

Zabbix SNMP Integration: <https://www.zabbix.com/integrations/snmp>

Zabbix SNMP Agent Documentation:

<https://www.zabbix.com/documentation/current/en/manual/config/items/itemtypes/snmp>

Zabbix Supported Trigger Functions:

<https://www.zabbix.com/documentation/current/en/manual/appendix/functions>

Zabbix GitHub oficial: <https://github.com/zabbix/zabbix>

OpenAI - ChatGPT: <https://chatgpt.com/>

Anthropic - Claude: <https://claude.ai/>

Annex d'enllaços del repositori Hefesto

Aquest annex recull els enllaços principals als README.md de cada mini projecte del repositori, per facilitar la navegació des de la documentació final.

Repositori base: [dpanero/Projecte_Hefesto_DPJ_ADR](#)

Nom del mini projecte	Descripció breu
09_Proxmox	Documentació de l'entorn de virtualització Proxmox utilitzat com a base del laboratori Hefesto.
04_IDS_IPS_Suricata	Documentació del sistema IDS/IPS amb Suricata, orientat a la detecció, la prevenció i l'anàlisi de seguretat.
05_Monitorització_de_xarxes_Zabbix	Documentació de la monitorització de xarxa i infraestructura mitjançant Zabbix.
01_Docker_Orquestradors_Basic	Documentació del desplegament de microserveis i proves amb Docker, Docker Compose, Swarm i Kubernetes.